

Struktur Berkait

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

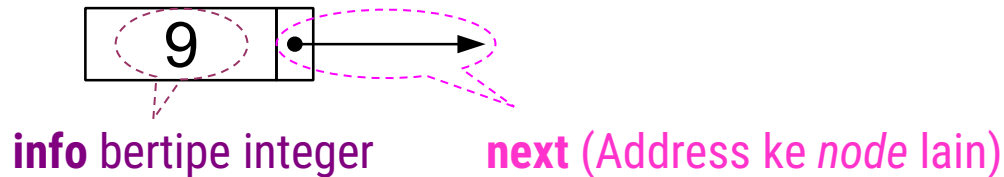
Definisi

Struktur berkait terdiri atas *node* yang terkait dengan *node* lain.

Node merupakan sebuah *tuple* yang terdiri atas dua bagian:

- 1) Sebuah nilai dengan tipe tertentu (info),
- 2) Sebuah penunjuk ke *node* lain (next).
Bisa jadi tidak menunjuk ke mana pun (NIL).

Ilustrasi:



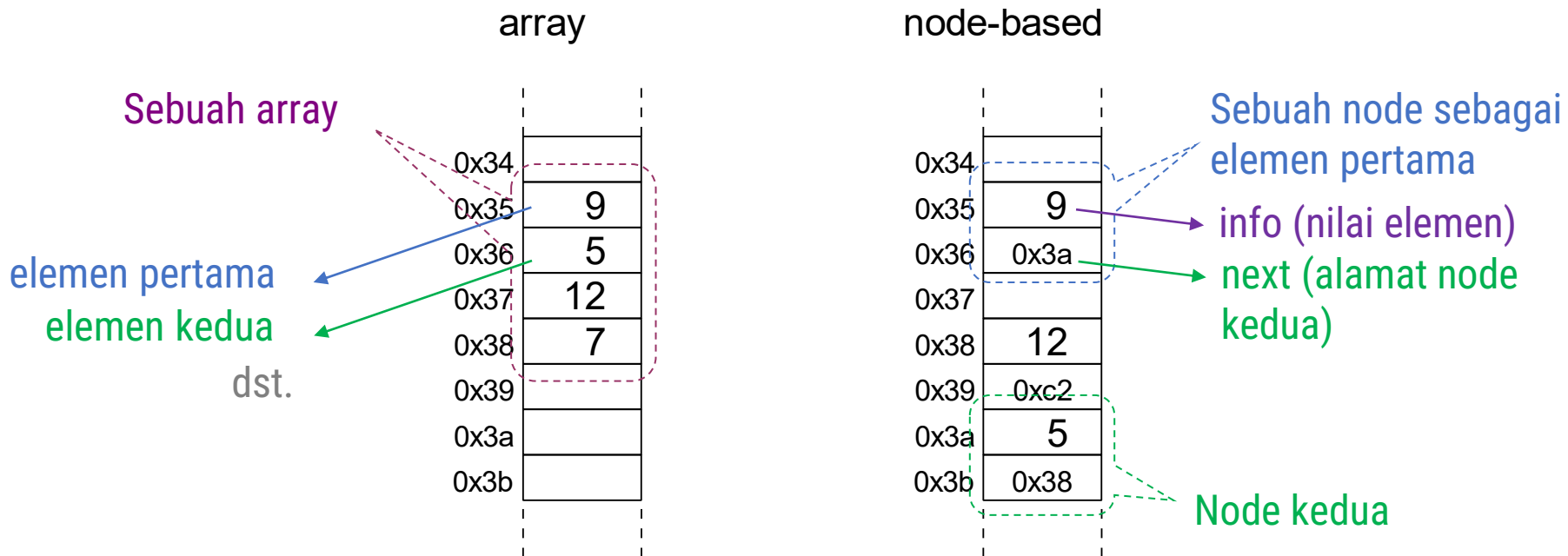
Hal ini memungkinkan penyimpanan elemen-elemen tanpa harus kontigu.

Disebut juga struktur *node-based*, *linked list*, atau *linear list*.

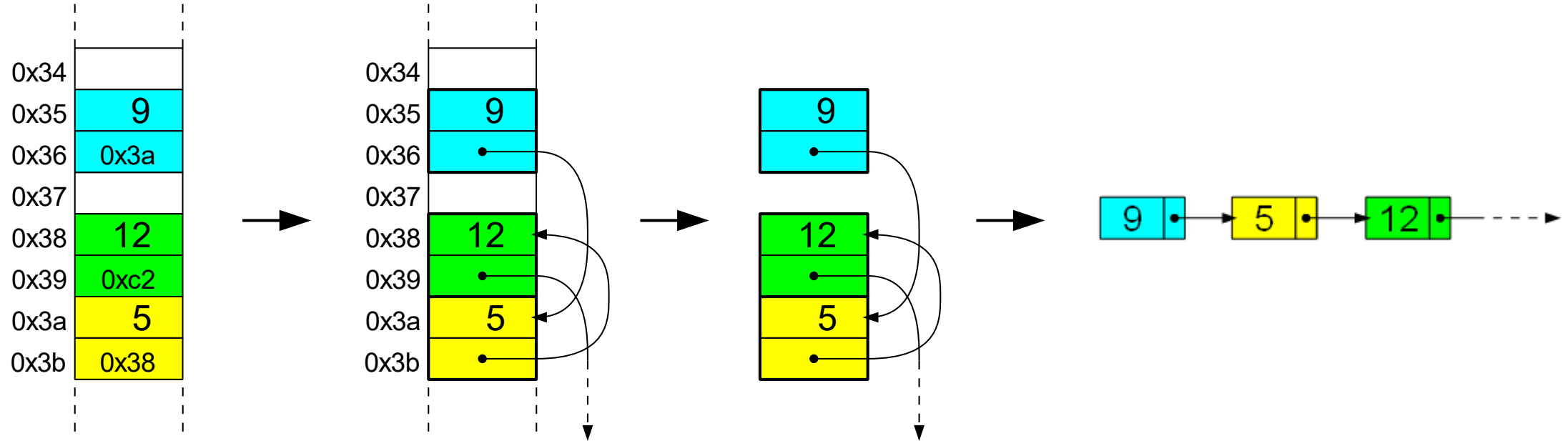
Memori fisik: array vs. node-based

- **Array:** elemen-elemen berada pada lokasi bersebelahan.
- **Node:** “next” mencatat alamat elemen berikutnya.

Contoh, dengan elemen bertipe *byte* (1 elemen mengisi 1 slot memori):



Ilustrasi struktur berkait



Karakteristik struktur data berbasis node

Memori dialokasi sesuai kebutuhan.

- Jika ada 3 elemen maka hanya perlu memori sebesar ukuran *node* × 3.
- Berbeda dengan *array* yang, misalnya sudah dialokasikan 100 elemen maka menggunakan memori sebesar ukuran elemen × 100 meskipun pada suatu waktu hanya 3 elemen yang efektif.

Ukuran memori per elemen menjadi lebih besar.

Ukuran elemen + ukuran *pointer*.

Secara umum mengorbankan efisiensi ruang (memori) demi efisiensi waktu.

(Tidak untuk semua jenis operasi.)

Node dalam Notasi Algoritmik

{ Deklarasi tipe bentukan }

type ElType: integer

type Address: pointer to Node

type Node: < info: ElType,
 next: Address >

{ Deklarasi variabel }

p1: Address

p2: Address

{ Inisialisasi dan penggunaan variabel }

p1 ← alokasi(9) { p1 menunjuk ke Node dengan info=9 dan next=NIL }

p2 ← alokasi(5) { p2 menunjuk ke Node dengan info=5 dan next=NIL }

p1↑.next ← p2 { Address next pada p1 menunjuk ke node yang ditunjuk p2 }

Node dalam Bahasa C (deklarasi tipe bentukan)

```
/* node.h */
#ifndef NODE_H
#define NODE_H

typedef int ElType;
typedef struct node* Address;
typedef struct node {
    ElType info;
    Address next;
} Node;

#define INFO(p) (p)->info
#define NEXT(p) (p)->next
Address newNode(ElType val);

#endif
```

```
/* node.c */
#include "node.h"
#include <stdlib.h>

Address newNode(ElType val) {
    Address p = (Address)
                malloc(sizeof(Node));
    if (p!=NULL) {
        INFO(p) = val;
        NEXT(p) = NULL;
    }
    return p;
}
```

Node dalam Bahasa C (contoh penggunaan)

```
/* Deklarasi variabel */
```

```
Address p1, p2;
```

```
/* Inisialisasi dan penggunaan variabel */
```

```
p1 = newNode(9); /* p1 menunjuk ke Node dengan info=9 dan next=NIL */
```

```
p2 = newNode(5); /* p2 menunjuk ke Node dengan info=5 dan next=NIL */
```

```
NEXT(p1) = p2; /* Address next pada p1 menunjuk ke node yang ditunjuk p2 */
```

Representasi Fisik List Linier: Struktur Berkait dengan Pointer

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Representasi implisit dan eksplisit

Sebelumnya kita telah menjumpai list berkait dengan representasi **implisit**

yaitu struktur data list berkait di mana *menunjuk ke sebuah list* adalah sama dengan *menunjuk ke elemen pertamanya*:

```
type ElType: ...  
type Address: pointer to Node  
type Node: < info: ElType,  
              next: Address >  
type List: Address
```

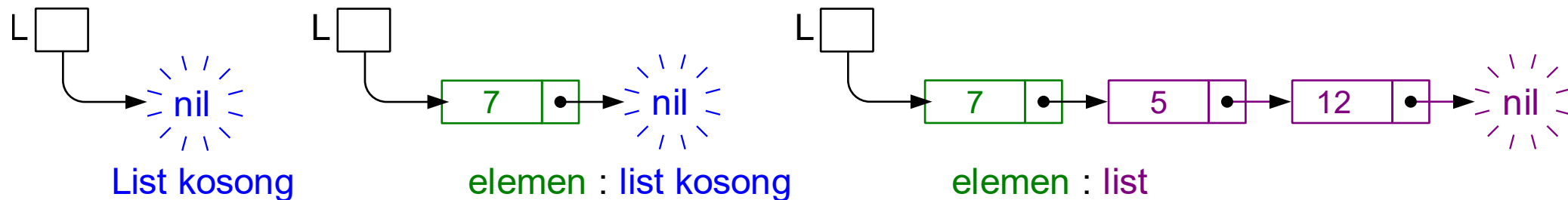
Terdapat representasi lain yaitu **eksplisit**, di mana elemen pertama list merupakan *bagian dari* struktur data list:

```
type List: < first: Address >
```

Representasi implisit

Representasi implisit mewakili definisi rekursif sebuah list linier seperti dalam paradigma fungsional:

- List kosong adalah list.
- List tidak kosong terdiri atas sebuah elemen yang diikuti list.



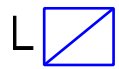
Elemen pertama list L, First = L.

Next dari First harus merupakan list juga, $\therefore$ type List: Address

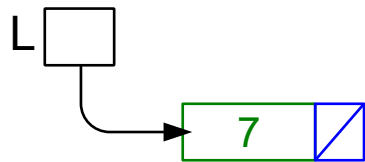
Representasi implisit

Representasi implisit mewakili definisi rekursif sebuah list linier seperti dalam paradigma fungsional:

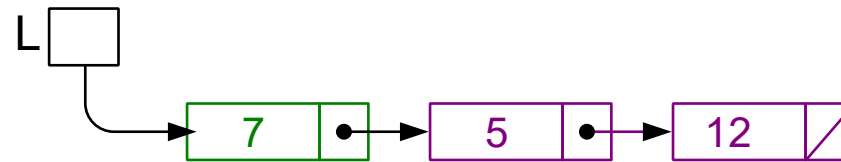
- List kosong adalah list.
- List tidak kosong terdiri atas sebuah elemen yang diikuti list.



List kosong



elemen : list kosong



elemen : list

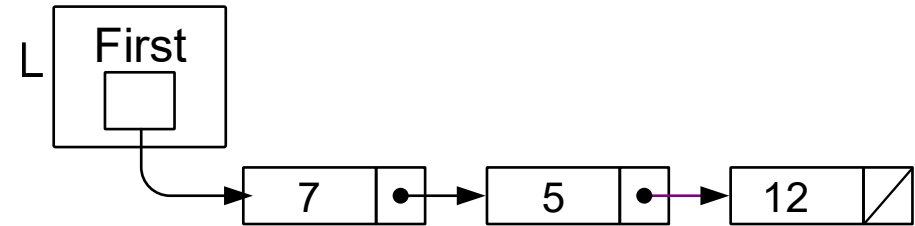
Elemen pertama list L, First = L.

Next dari First harus merupakan list juga, $\therefore$ type List: Address

Representasi eksplisit

Dalam representasi eksplisit, First merupakan *bagian* dari struktur data list.

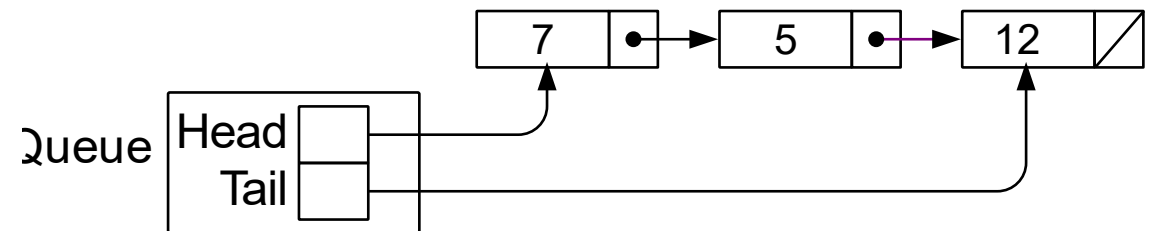
```
type List: < first: Address >
```



∴ akses ke elemen pertama 1 adalah 1.first.

Representasi ini berguna misalnya pada implementasi Queue memanfaatkan struktur list berkait:

```
type Queue: < head: Address,  
              tail: Address >
```



Proses rekursif

```
type ElType: ...  
type Address: pointer to Node  
type Node: < info: ElType,  
             next: Address >  
type List: Address
```

{ Representasi Implisit }



```
procedure displayList(input l: List)  
{ I.S. l terdefinisi  
  F.S. Setiap elemen l di-print }
```

KAMUS LOKAL

ALGORITMA

```
if (isEmpty(l)) then { Basis 0 }  
  { tidak melakukan apa-apa }  
else { Rekurens }  
  output(l↑.info)  
  PrintList(l↑.next)
```

...

```
displayList(l)
```

Proses rekursif

```
type ElType: ...  
type Address: pointer to Node  
type Node: < info: ElType,  
             next: Address >  
type List: < first: Address >
```

{ Representasi Eksplisit }



*{ PROBLEMS:
 l tidak memiliki info.
 l tidak memiliki next. }*

```
procedure displayList(input l: List)  
{ I.S. l terdefinisi  
  F.S. Setiap elemen list diprint }
```

KAMUS LOKAL

ALGORITMA

```
if (isEmpty(l)) then { Basis 0 }  
  { tidak melakukan apa-apa }  
else { Rekurens }  
  output(l↑.info)  
  PrintList(l↑.next)
```

...

displayList(l)

Proses rekursif

```
type ElType: ...  
type Address: pointer to Node  
type Node: < info: ElType,  
             next: Address >  
type List: < first: Address >
```

{ Representasi Eksplisit }



*{ PROBLEM:
l.first↑.next merupakan Address,
tidak bisa di-pass ke displayList yang
menerima sebuah List. }*

```
procedure displayList(input l: List)  
{ I.S. l terdefinisi  
  F.S. Setiap elemen list diprint }
```

KAMUS LOKAL

ALGORITMA

```
if (isEmpty(l)) then { Basis 0 }  
  { tidak melakukan apa-apa }  
else { Rekurens }  
  output(l.first↑.info)  
  PrintList(l.first↑.next)
```

...

```
displayList(l)
```

Proses rekursif

```
type ElType: ...  
type Address: pointer to Node  
type Node: < info: ElType,  
             next: Address >  
type List: < first: Address >
```

{ Representasi Eksplisit }



*{ PROBLEM:
p merupakan Address, tidak bisa di-pass
ke isEmpty yang menerima sebuah List. }*

```
procedure displayList (input p: Address)  
{ I.S. p terdefinisi  
  F.S. Setiap elemen list diprint }
```

KAMUS LOKAL

ALGORITMA

```
if (isEmpty(p)) then { Basis 0 }  
  { tidak melakukan apa-apa }  
else { Rekurens }  
  output(p↑.info)  
  displayList(p↑.next)
```

...

```
displayList(l.first)
```

Representasi Fisik List Linier: Struktur Berkait dengan Array

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Kekurangan struktur berkait

Pada pembahasan implementasi ADT List menggunakan struktur berkait, setiap Node dialokasikan satu demi satu.

- Persoalan: alokasi & dealokasi memori adalah operasi yang “mahal” pada sistem operasi.
- Akan lebih efisien jika dapat dilakukan alokasi beberapa Node sekaligus.

Persoalan lain: bahasa pemrograman yang digunakan mungkin tidak mendukung pointer.

Alternatif: array of Node

Banyak Node dialokasi dengan satu kali pemanggilan ke sistem operasi, dalam bentuk array.

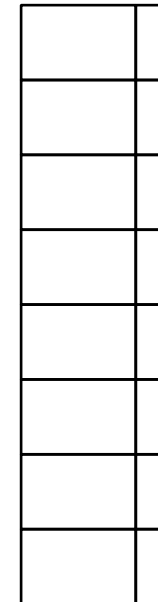
Bagian Next dari Node kini bukan mengacu pada alamat fisik memori melainkan indeks array.

Array of Node dapat dideklarasikan secara global untuk digunakan oleh beberapa List sekaligus.

Node



array of Node



Array of Node

Saat inisialisasi, bagian Next setiap Node diisi dengan indeks elemen array berikutnya (`nodeArray[i].next = i+1`).

Untuk Node terakhir, diisi dengan indeks yang tidak valid (konstanta, misal -1).

Diperlukan sebuah pencatat Node pertama yang kosong.

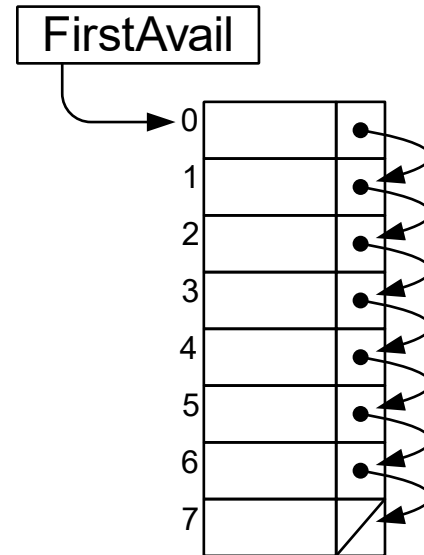
Saat inisialisasi, diisi dengan indeks pertama array (0).

Ilustrasi: setelah inisialisasi

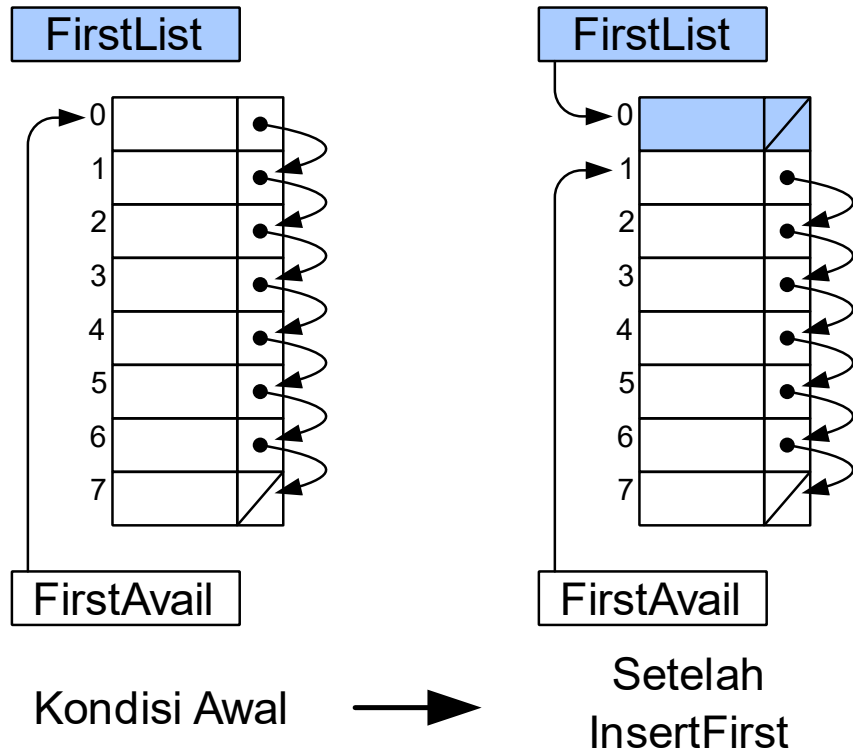
FirstAvail = 0

0		1
1		2
2		3
3		4
4		5
5		6
6		7
7		-1

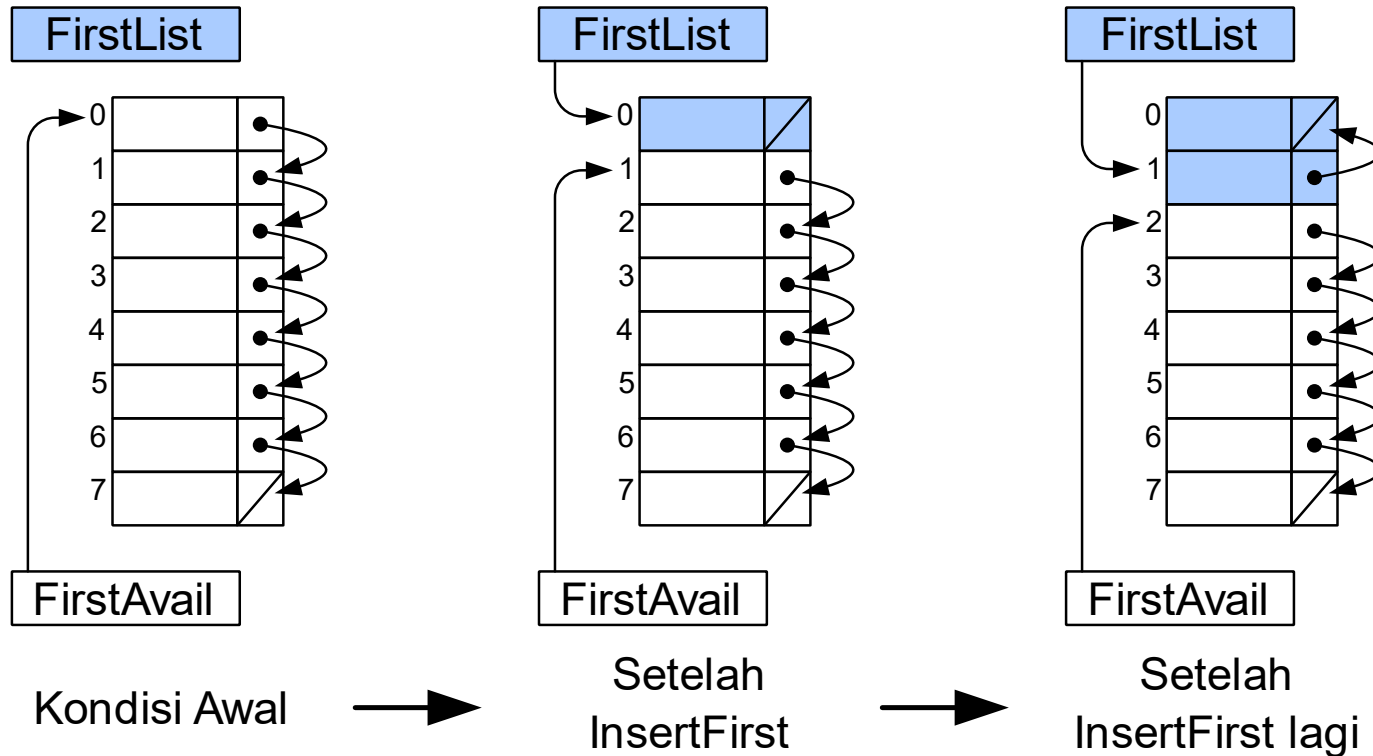
atau



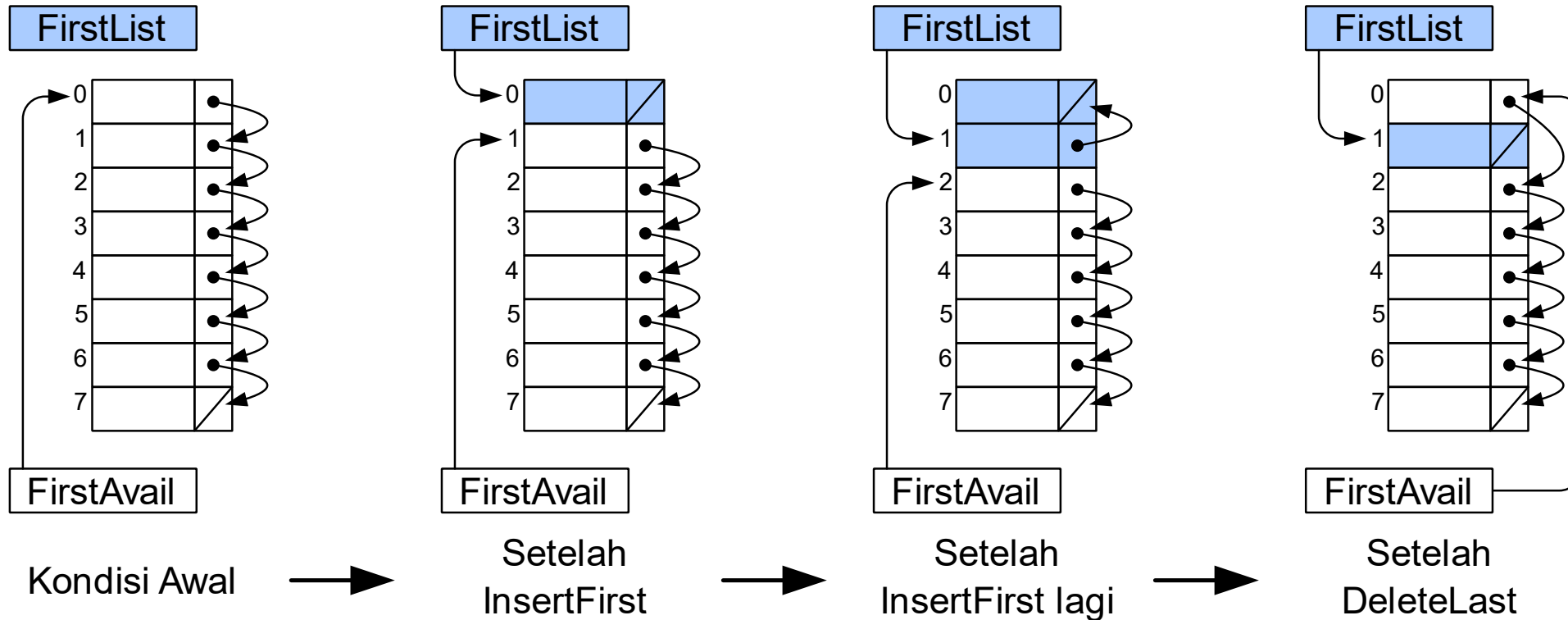
Ilustrasi: pemakaian memori list



Ilustrasi: pemakaian memori list



Ilustrasi: pemakaian memori list



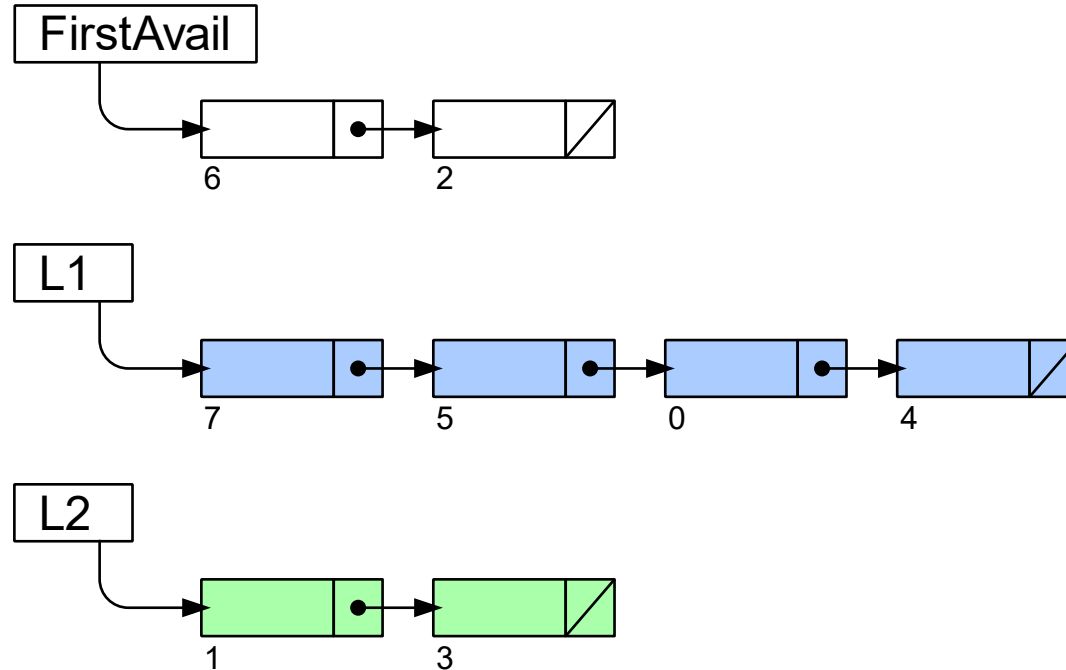
Ilustrasi: array digunakan oleh dua list

FirstAvail = 6

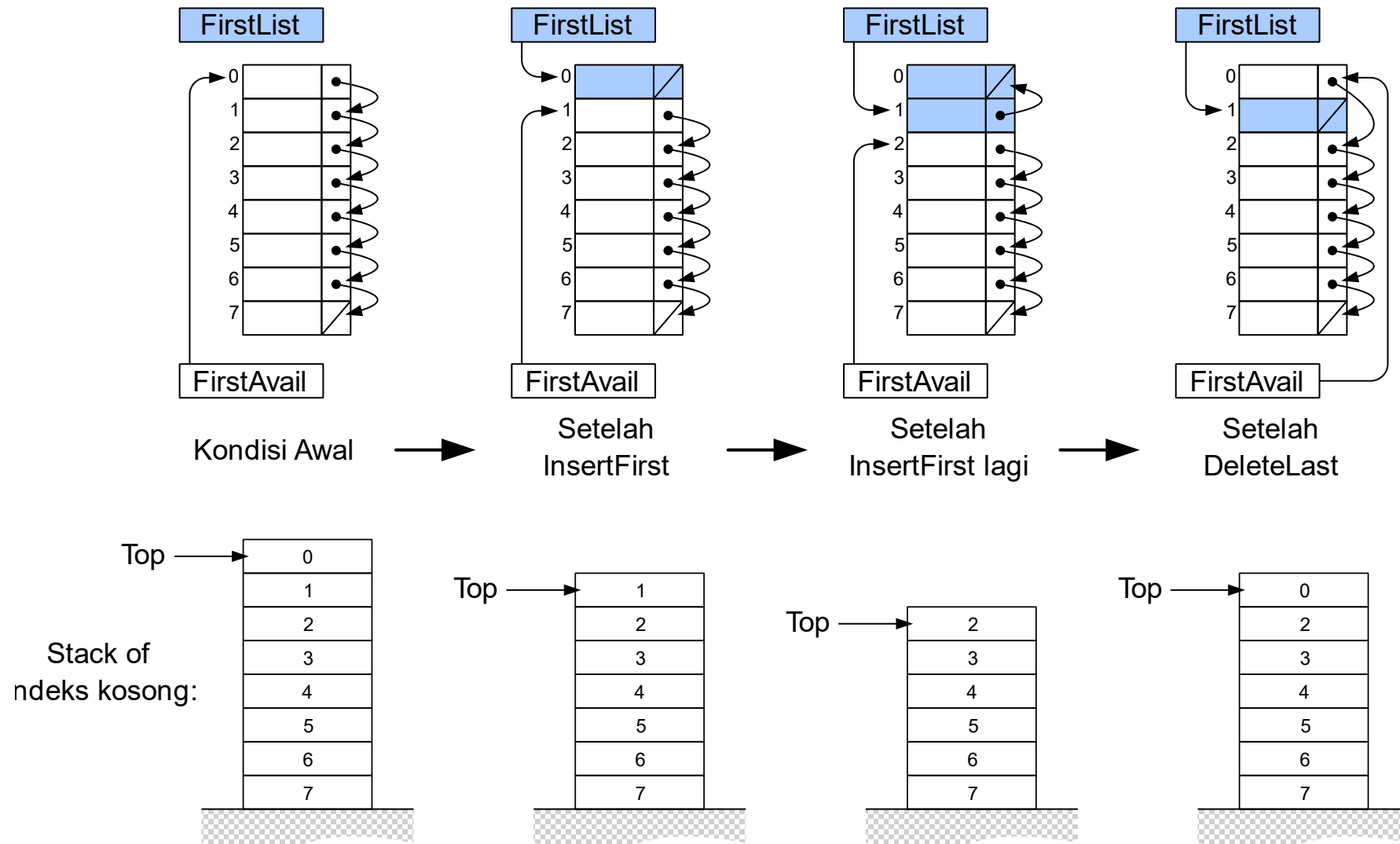
First L1 = 7

First L2 = 1

0		4
1		3
2		/
3		/
4		/
5		0
6		2
7		5



Indeks yang kosong membentuk sebuah Stack!



Latihan Soal ADT List dengan Struktur Berkait

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Latihan Soal

1. Buatlah fungsi **countPos** yang menghitung banyaknya kemunculan bilangan positif (>0) dari sebuah list of integer l

function countPos(l: List) $\rightarrow$ integer

2. Buatlah fungsi **max** yang menghasilkan nilai maksimum dari suatu list of integer l yang tidak kosong

function max(l: List) $\rightarrow$ integer

3. Buatlah fungsi **searchPos** yang menghasilkan address di mana nilai positif pertama kali ditemukan di list of integer l

function searchPos(l: List) $\rightarrow$ address

Latihan Soal

4. Buatlah prosedur **deleteNeg** yang menghapus semua elemen bernilai negatif (<0) pada sebuah list of integer l . List l boleh kosong dan setiap elemen yang dihapus harus dilakukan dealokasi.

procedure deleteNeg(input/output l :List)

5. Buatlah prosedur **copyPos** yang menyalin semua elemen bernilai positif (>0) dari sebuah list of integer $l1$ menjadi $l2$

procedure copyPos(input $l1$:List, output $l2$:List)

6. Buatlah prosedur **sortedInsert** yang menambahkan sebuah elemen x pada sebuah list of integer l yang terurut menaik

procedure sortedInsert(input/output l :List, input x :ElType)

Latihan Soal

7. Buatlah prosedur **updateList** yang menerima sebuah infotype x dan y dan sebuah list l dan kemudian mengganti elemen pertama l yang bernilai x dengan y jika x ada di l. Jika x tidak ada di l, l tetap.

```
procedure updateList(input x,y: infotype, input/output l: List)
```

Variasi Struktur List Linier

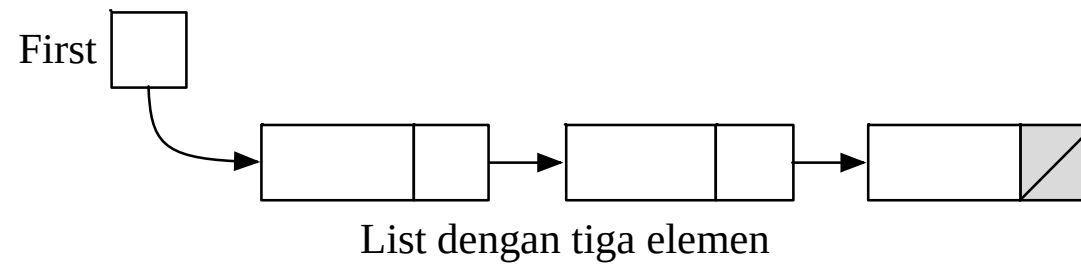
IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

List Linier

Elemen pertama ???

Elemen terakhir ???

List kosong ???

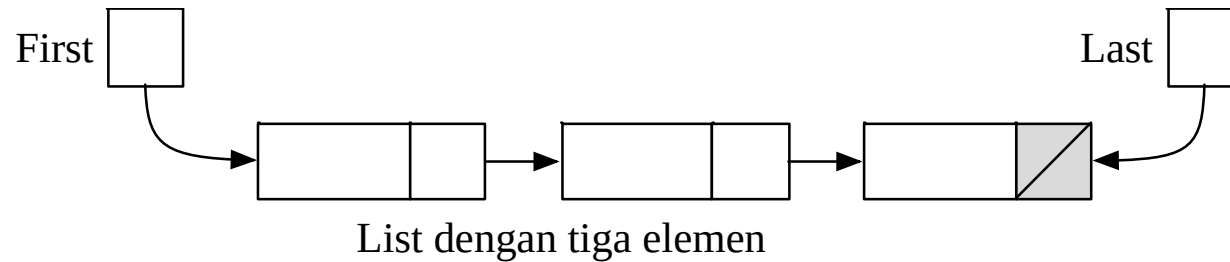


List Linier yang dicatat alamat elemen pertama dan terakhir

Elemen pertama ????

Elemen terakhir ???

List kosong ???

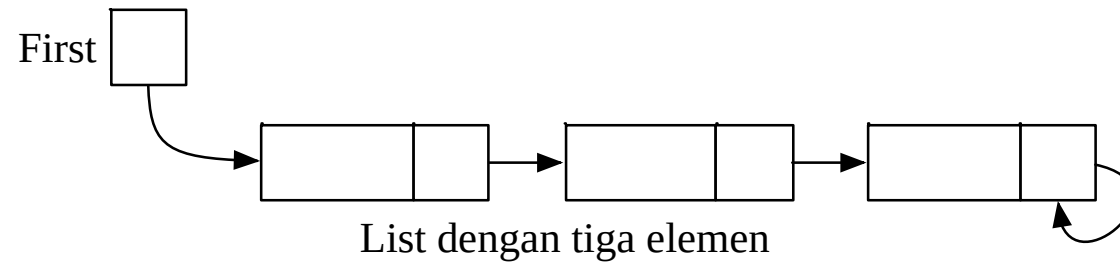


List yang Elemen Terakhir Menunjuk pada Diri Sendiri

Elemen pertama ????

Elemen terakhir ???

List kosong ???

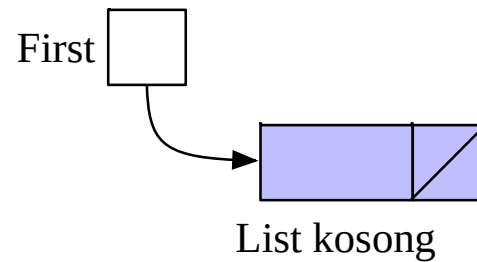
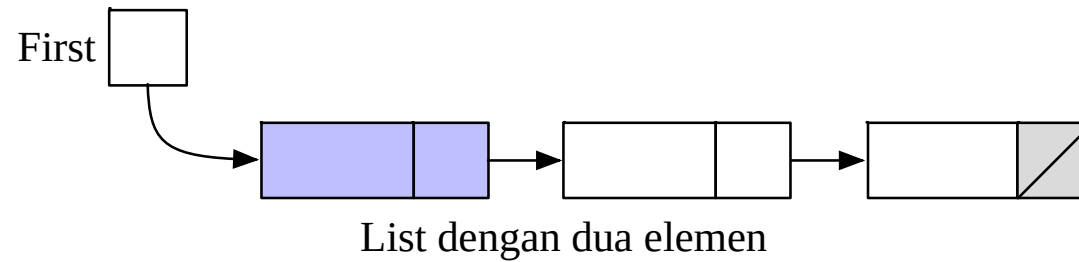


List dengan “*Dummy Element*” pada Elemen Pertama

Elemen pertama ????

Elemen terakhir ???

List kosong ???

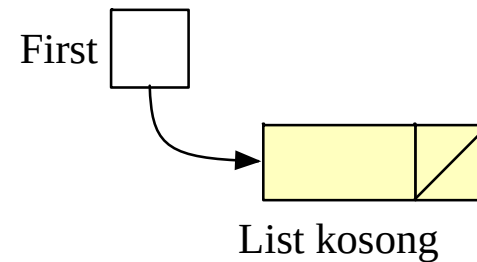
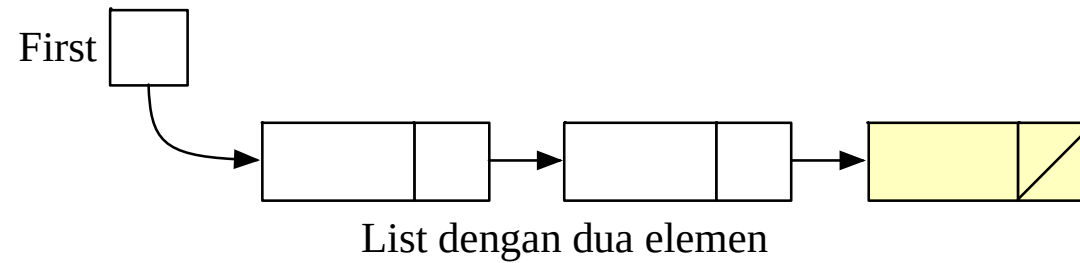


List dengan “*Dummy Element*” pada Elemen Terakhir

Elemen pertama ????

Elemen terakhir ???

List kosong ???

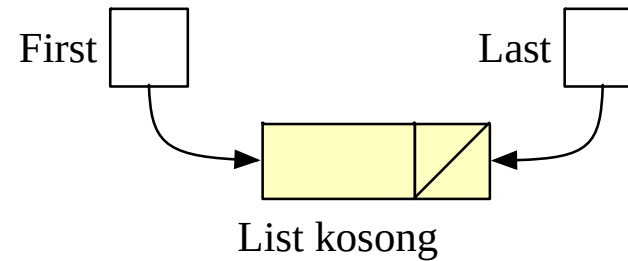
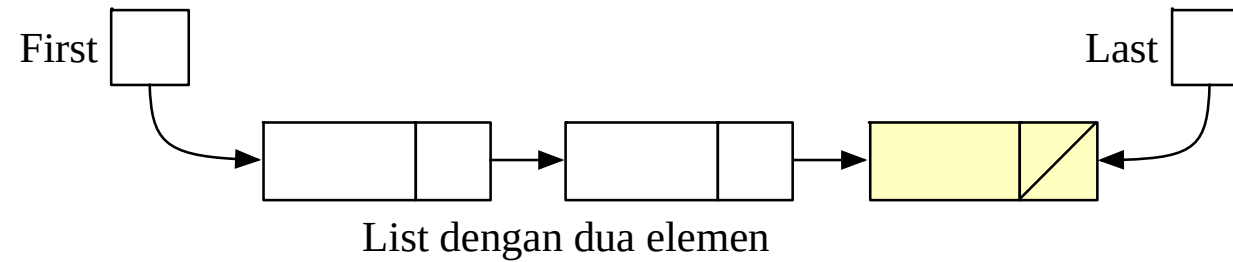


List dengan *Dummy Element* di Akhir dan Pencatatan Alamat Elemen Akhir

Elemen pertama ????

Elemen terakhir ???

List kosong ???

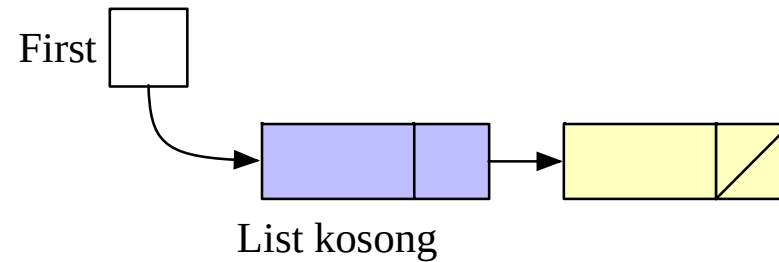
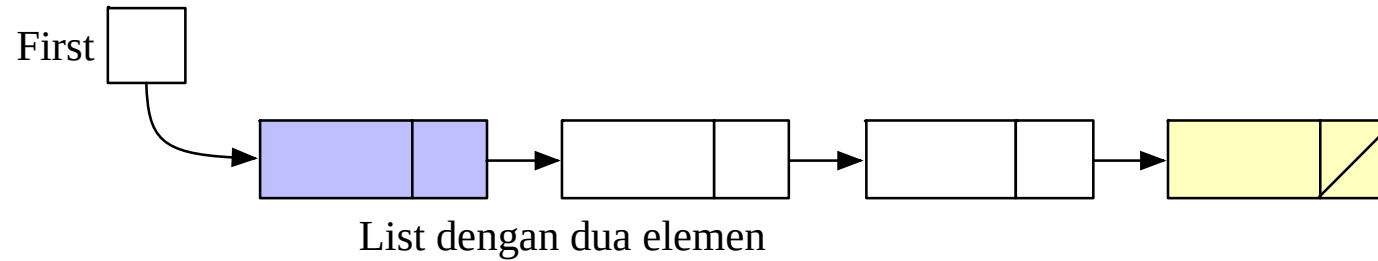


List dengan *Dummy Element* pada Elemen Pertama dan Terakhir

Elemen pertama ????

Elemen terakhir ???

List kosong ???

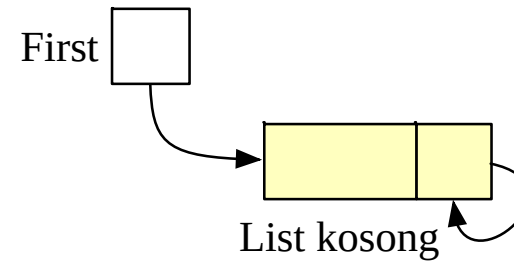
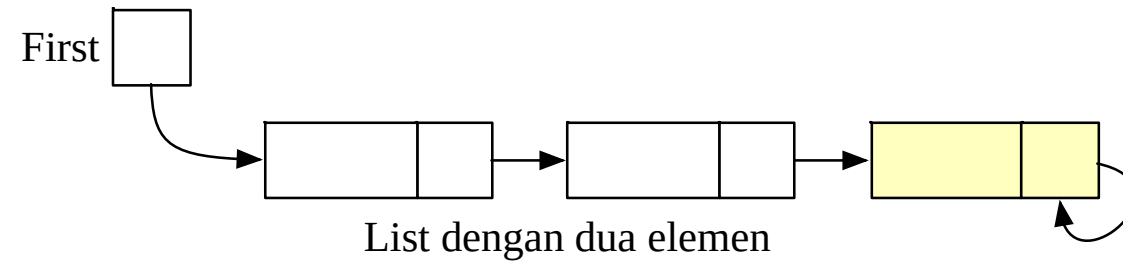


List dengan *Dummy Element* pada Elemen Terakhir yang Menunjuk ke Diri Sendiri

Elemen pertama ????

Elemen terakhir ???

List kosong ???

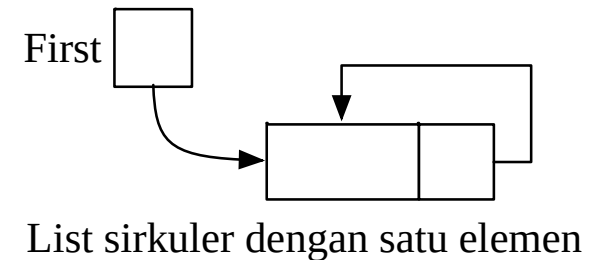
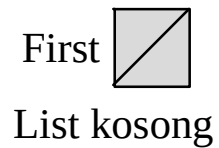
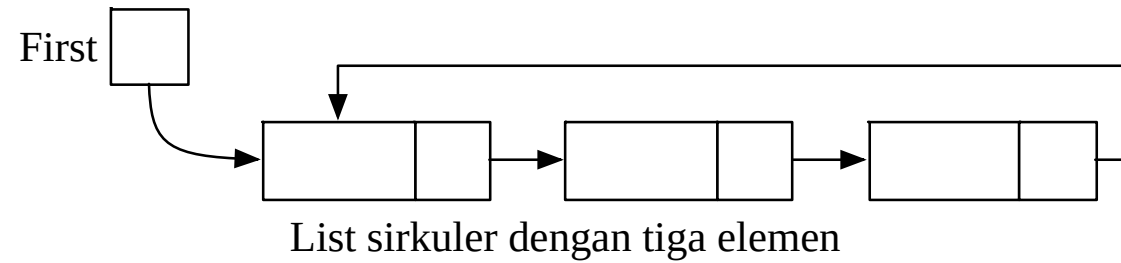


List Sirkuler

Elemen pertama ????

Elemen terakhir ???

List kosong ???

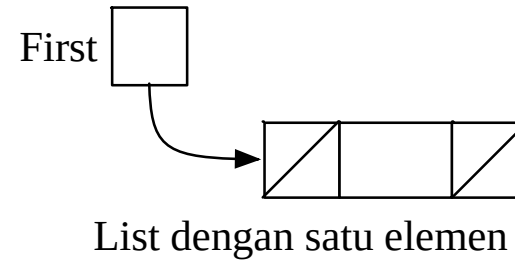
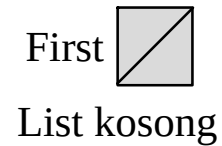
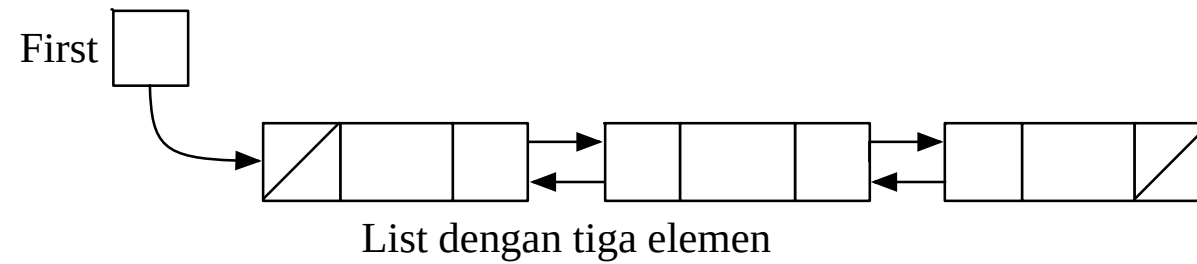
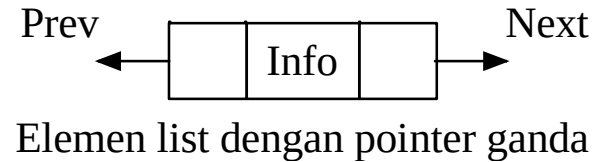


List dengan Pointer Ganda (*doubly linked list*)

Elemen pertama ????

Elemen terakhir ???

List kosong ???

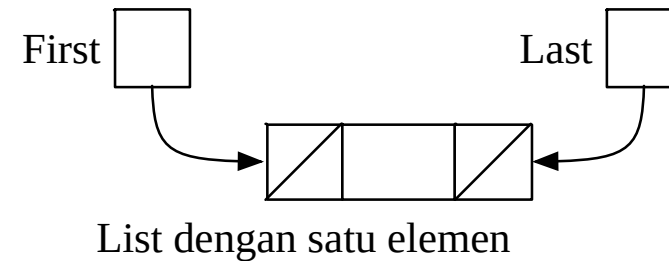
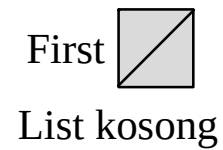
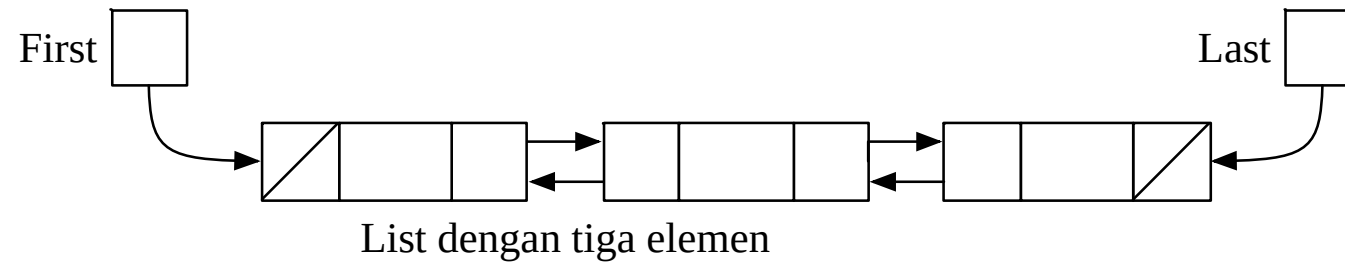


List dengan Pointer Ganda dan Pencatatan Alamat Elemen Terakhir

Elemen pertama ????

Elemen terakhir ???

List kosong ???

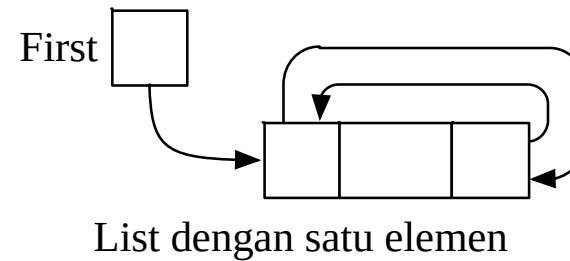
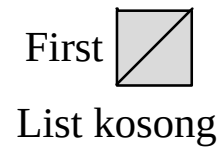
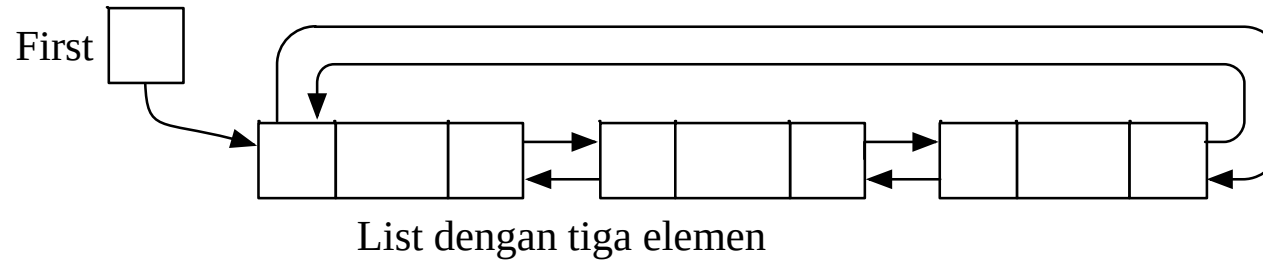


List dengan Pointer Sirkuler Ganda

Elemen pertama ????

Elemen terakhir ???

List kosong ???



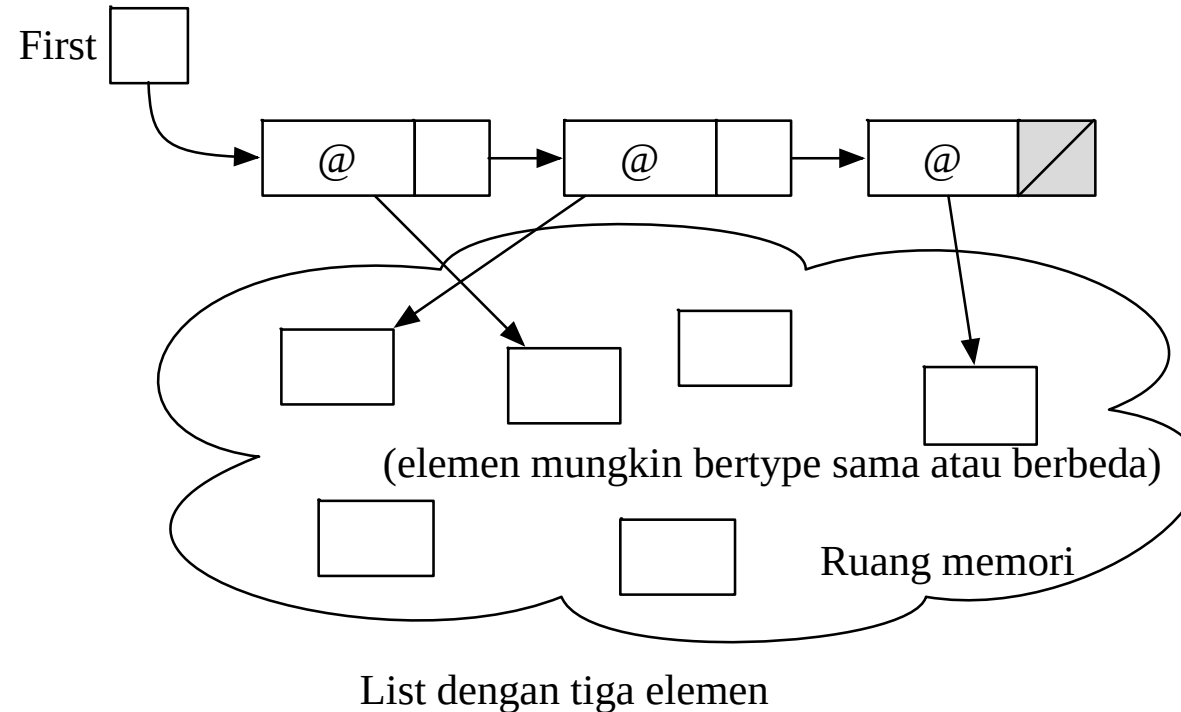
List yang Informasi Elemennya adalah Alamat dari Suatu Informasi

Elemen pertama ????

Elemen terakhir ???

List kosong ???

First 
List kosong



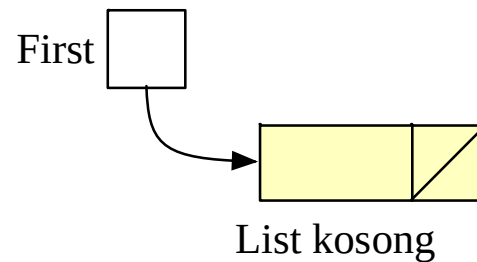
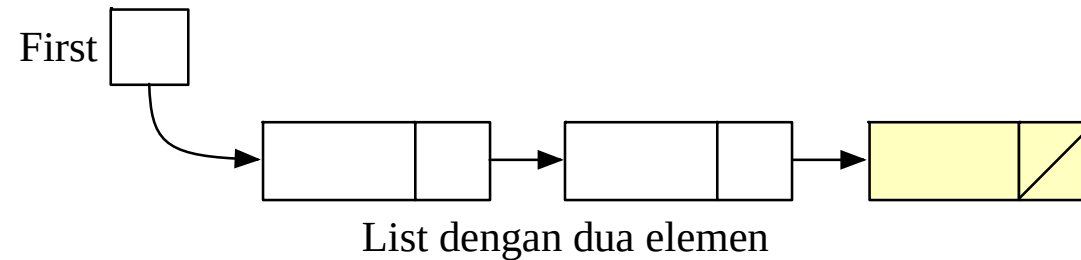
List dengan *dummy element* di akhir

List dengan “*Dummy Element*” pada Elemen Terakhir

Elemen pertama: $\text{First}(l) = l$

Elemen terakhir: ber-address p , dengan $\text{Next}(p) = \text{dummy}@$

List kosong: $\text{First}(l) = \text{dummy}@$

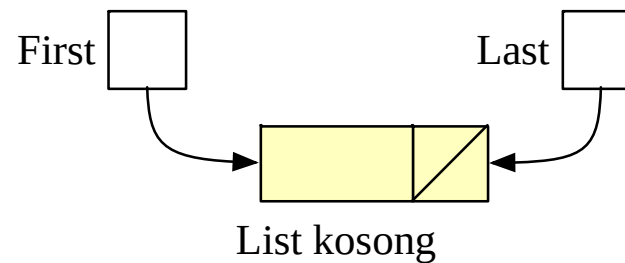
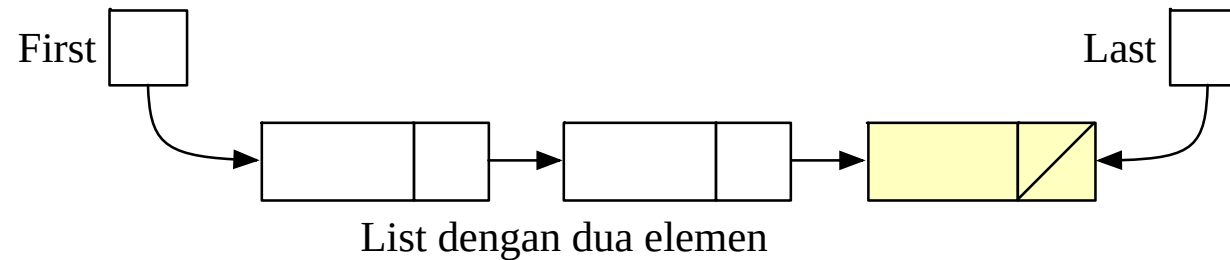


List dengan *Dummy Element* di Akhir dan Pencatatan Alamat Elemen Akhir

Elemen pertama: $\text{First}(l) = l$

Elemen terakhir: ber-address p , dengan $\text{Next}(p) = \text{dummy@}$

List kosong: $\text{First}(l) = \text{Last}(l) = \text{dummy@}$



Beberapa catatan

Sering dipakai jika *dummy* adalah sentinel, dan pencarian diperlukan sebelum penambahan elemen

- Nilai yang dicari dapat secara langsung disimpan untuk sementara pada *dummy*, kemudian dilakukan *search*
- Jika *search* tidak berhasil, dan elemen akan ditambahkan, maka dialokasi sebuah *dummy* yang baru, nilai Last berubah
- Contoh pemakaian dijelaskan pada *topological sort* (Lihat Studi Kasus 6)

Dummy bisa berupa address yang tetap, bisa sebuah address yang berbeda (setiap kali *dummy* tersebut dipakai sebagai elemen list, dialokasi *dummy* yang baru)

Variasi List Linier

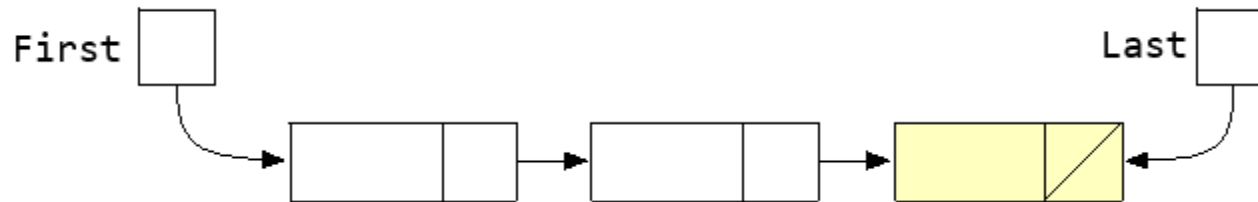
List Linier dengan Dummy Element

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Bahasan

Buatlah ADT list + driver dengan elemen dummy di akhir:

- Penunjuk first dan last (last element menunjuk ke elemen dummy)
- Alamat dummy: tetap
- Representasi fisik: berkait dengan pointer



Representasi Fisik dengan Pointer

```
#define NIL NULL
#define IDX_UNDEF -1
typedef int ElType;
typedef struct node* Address;
typedef struct node { ElType info; Address next; } Node;

/* Definisi list: */
/* List kosong: First(L) = Last(L) = dummy@ */
/* Setiap elemen dengan address P dapat diacu Info(P), Next(P) */
/* Elemen dummy terletak pada last */
typedef struct {
    address first;
    address last;
} List;

/* Selektor */
#define INFO(P) (P)->info
#define NEXT(P) (P)->next
#define FIRST(L) ((L).first)
#define LAST(L) ((L).last)
```

Beberapa primitif

Buatlah sebagai latihan:

```
/* PROTOTYPE */
/***** TEST LIST KOSONG *****/
boolean isEmpty(List l);
/* Mengirim true jika list kosong: FIRST(l) = dummy@
   dan LAST(L) = dummy@ */
/***** PEMBUATAN LIST KOSONG *****/
void CreateList(List *l);
/* I.S. sembarang */
/* F.S. Terbentuk list l kosong, dengan satu elemen dummy */
/*      Jika gagal maka FIRST(l) = LAST(l) = NIL dan list gagal terbentuk */
/***** SEARCHING *****/
int indexOf(List l, ElType x);
/* Mengembalikan indeks node list dengan nilai X, atau IDX_UNDEF jika tidak ada */
```

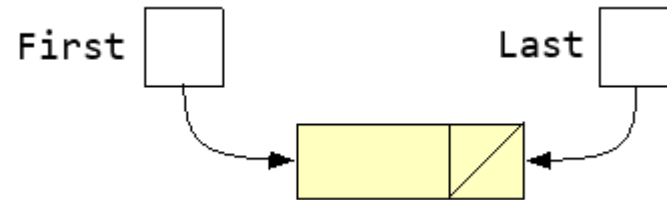
Beberapa primitif

Buatlah sebagai latihan:

```
void insertFirst(List *l, ElType x);
/* I.S. List l terdefinisi */
/* F.S. Menambahkan elemen x sebagai elemen pertama List l */
void insertLast(List *l, ElType x);
/* I.S. List l terdefinisi */
/* F.S. x ditambahkan sebagai elemen terakhir l yang baru, */
/*      yaitu menjadi elemen sebelum elemen dummy */
void deleteFirst(List *l, ElType *x);
/* I.S. List l tidak kosong */
/* F.S. x adalah nilai elemen pertama list l sebelum penghapusan */
/*      Elemen list berkurang satu (mungkin menjadi kosong) */
/*      First element yg baru adalah suksesor elemen pertama yang lama */
void deleteLast(List *l, ElType *x);
/* I.S. List l tidak kosong */
/* F.S. x adalah terakhir sebelum dummy pada list sebelum penghapusan */
/*      Elemen list berkurang satu (mungkin menjadi kosong) */
```

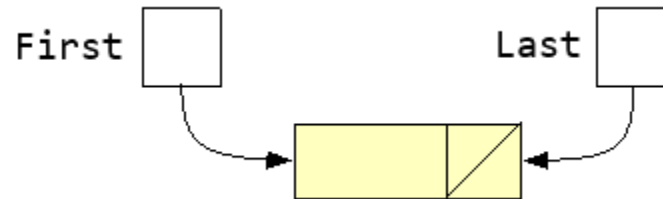
Beberapa primitif

```
boolean isEmpty(List l) {  
    /* Mengirim true jika list kosong */  
    /* Kamus Lokal */  
  
    /* Algoritma */  
    return (FIRST(l) == LAST(l));  
}
```



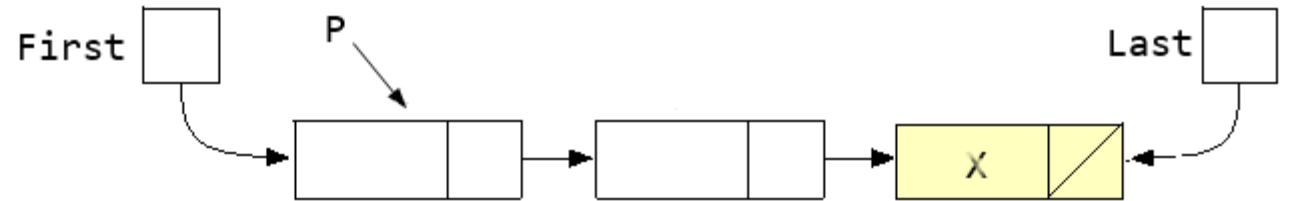
Beberapa primitif

```
void CreateList(List *l) {  
    /* I.S. Sembarang */  
    /* F.S. Terbentuk list l kosong, dengan satu elemen dummy,  
        jika alokasi dummy berhasil */  
    /* Jika gagal maka FIRST(l) = LAST(l) = NIL dan list gagal terbentuk */  
    /* Kamus Lokal */  
    address pDummy;  
  
    /* Algoritma */  
    pDummy = alokasi(0); /* INFO(pDummy) tidak didefinisikan */  
    if (Pdummy != Nil) {  
        FIRST(*L) = pDummy;  
        LAST(*L) = pDummy;  
    } else /* List gagal terbentuk */ {  
        FIRST(*L) = NIL;  
        LAST(*L) = NIL;  
    }  
}
```



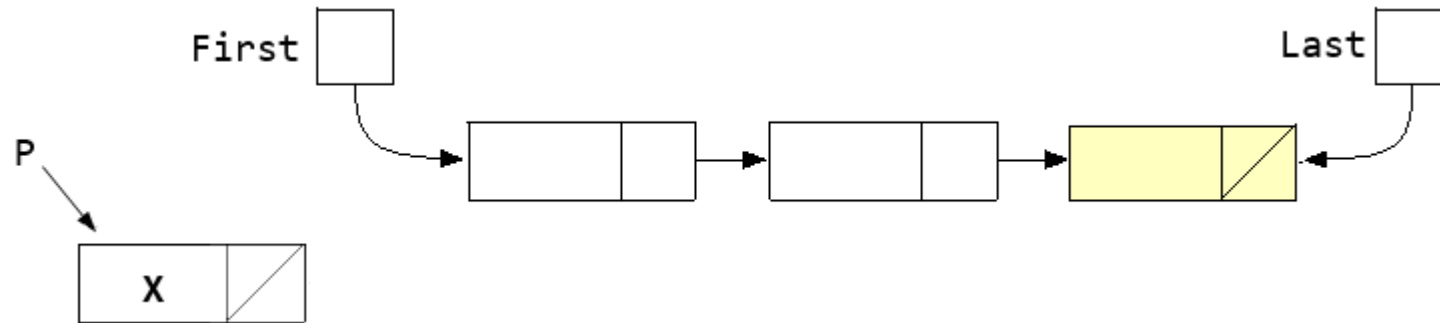
Beberapa primitif

```
int indexOf(List l, ElType x) {  
    /* Mengembalikan indeks node list dengan nilai X, atau IDX_UNDEF jika tidak ada */  
    /* Kamus Lokal */  
    address p;  
    int idx;  
    /* Algoritma */  
    INFO(LAST(l)) = x; /* letakkan X di sentinel */  
    p = FIRST(l);  
    idx = 0;  
    while (INFO(p) != x) {  
        p = NEXT(p);  
        idx++;  
    } /* INFO(p) = x */  
    if (p != LAST(l)) { /* bukan ketemu di sentinel */  
        return idx;  
    } else /* p = LAST(l), ketemu di sentinel */ {  
        return IDX_UNDEF;  
    }  
}
```



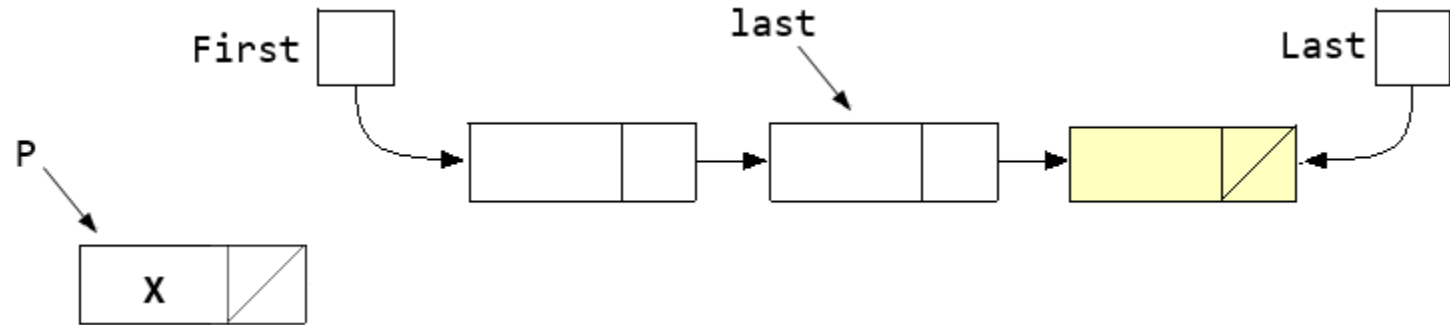
Beberapa primitif

```
void insertFirst(List *l, ElType x) {  
    /* I.S. List l terdefinisi */  
    /* F.S. Menambahkan elemen x sebagai elemen pertama List l */  
    /* Kamus Lokal */  
    address p;  
  
    /* Algoritma */  
    p = newNode(x);  
    if (p != NIL) {  
        NEXT(p) = FIRST(*l);  
        FIRST(*l) = p;  
    }  
}
```



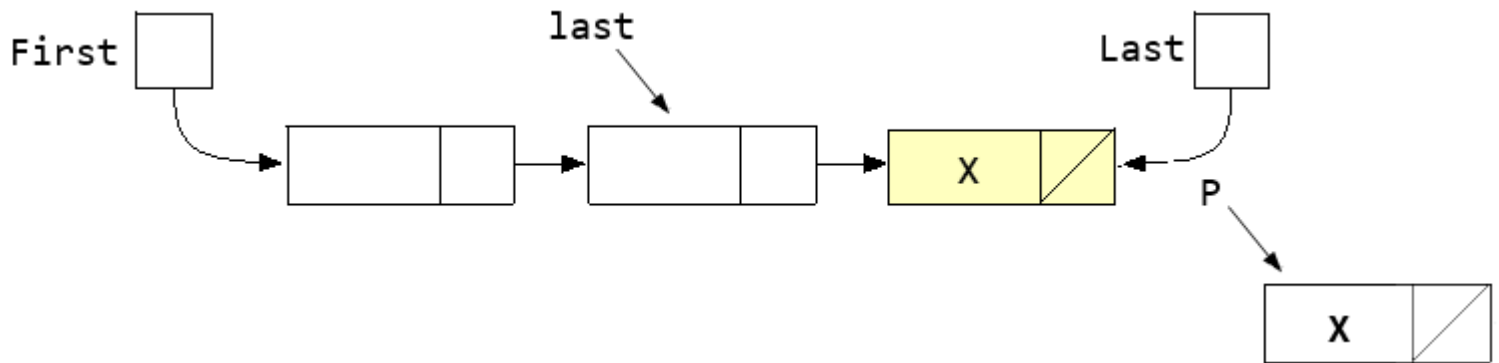
Beberapa primitif

```
void insertLast(List *l, ElType x) {  
    /* I.S. List l terdefinisi */  
    /* F.S. x ditambahkan sebagai elemen terakhir l yang baru */  
    /* Alamat elemen dummy tidak berubah */  
    /* Kamus Lokal */  
    address p, last;  
  
    /* Algoritma */  
    if (isEmpty(*l)) {  
        insertFirst(l,x);  
    } else {  
        p = newNode(x);  
        if (p != NIL) {  
            last = FIRST(*l);  
            while (NEXT(last) != LAST(*l)) {  
                last = NEXT(last);  
            } /* NEXT(last) == LAST(*l) alias dummy */  
            NEXT(last) = p;  
            NEXT(p) = LAST(*l);  
        }  
    }  
}
```



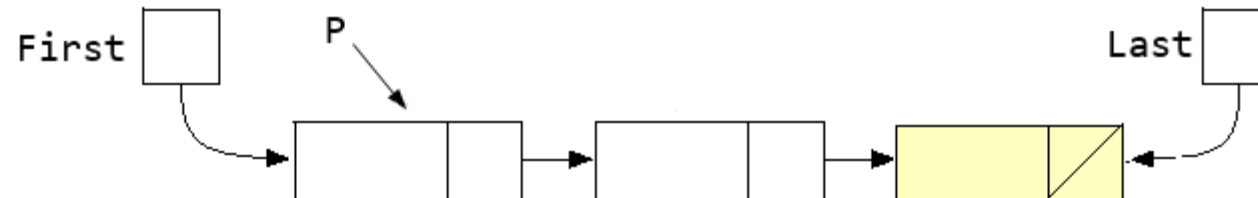
Beberapa primitif

```
void insertLast(List *l, ElType x) {  
    /* I.S. List l terdefinisi */  
    /* F.S. x ditambahkan sebagai elemen terakhir l yang baru */  
    /* Versi jika alamat elemen dummy boleh berubah */  
    /* Kamus Lokal */  
  
    /* Algoritma */  
    if (isEmpty(*l)) {  
        insertFirst(l,x);  
    } else {  
        INFO(LAST(*l)) = x;  
        p = newNode(x); /* dummy baru */  
        if (p != NIL) {  
            NEXT(LAST(*l)) = p;  
            LAST(*l) = p;  
        }  
    }  
}
```



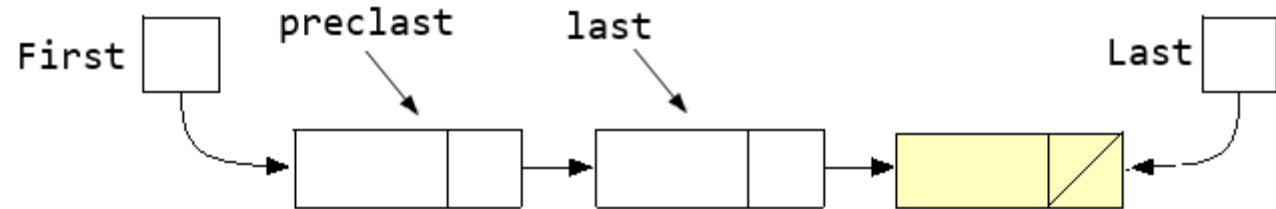
Beberapa primitif

```
void deleteFirst(List *l, ElType *x) {  
    /* I.S. List l tidak kosong */  
    /* F.S. x adalah elemen pertama list l sebelum penghapusan */  
    /*      Elemen list berkurang satu (mungkin menjadi kosong) */  
    /*      First element yg baru adalah suksesor elemen pertama yang lama */  
    /* Kamus Lokal */  
    address p;  
  
    /* Algoritma */  
    p = FIRST(*l);  
    *x = INFO(p);  
    FIRST(*l) = NEXT(FIRST(*l));  
    free(p);  
}
```



Beberapa primitif

```
void deleteLast(List *l, ElType *x) {  
    /* I.S. List l tidak kosong */  
    /* F.S. x adalah elemen terakhir list l sebelum penghapusan */  
    /*      Elemen list berkurang satu (mungkin menjadi kosong) */  
    /*      Last element baru adalah predesesor elemen terakhir yg lama, jika ada*/  
    /* Kamus Lokal */  
    address last, preLast;  
  
    /* Algoritma */  
    last = FIRST(*l); preLast = NIL;  
    while (NEXT(last) != LAST(*l)) {  
        preLast = last; last = NEXT(last);  
    }  
    *x = INFO(last);  
    if (preLast == NIL) { /* kasus satu elemen */  
        FIRST(*l) = LAST(*l);  
    } else {  
        NEXT(preLast) = LAST(*l);  
    }  
    free(last);  
}
```



Variasi List Linier

List dengan Pointer Ganda

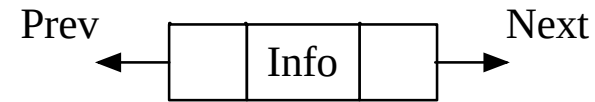
IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

List dengan Pointer Ganda

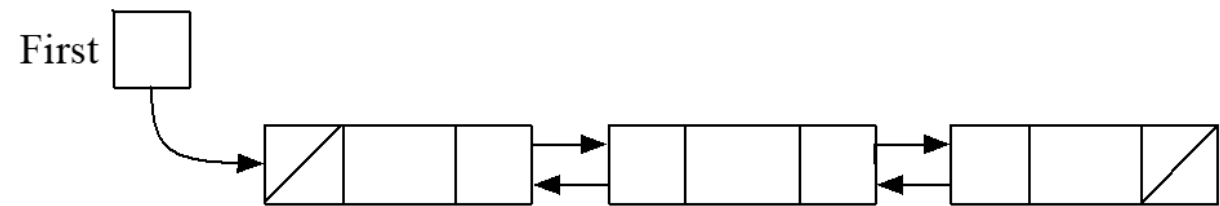
Elemen pertama: $\text{First}(L)$

Elemen terakhir: $\text{Next}(P) = \text{Nil}$

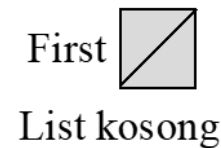
List kosong: $\text{First}(L) = \text{Nil}$



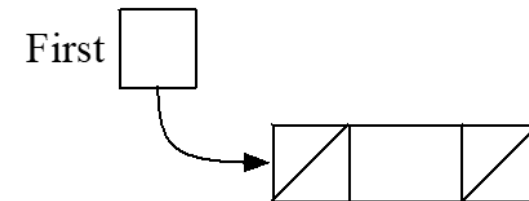
Elemen list dengan pointer ganda



List dengan tiga elemen



List kosong



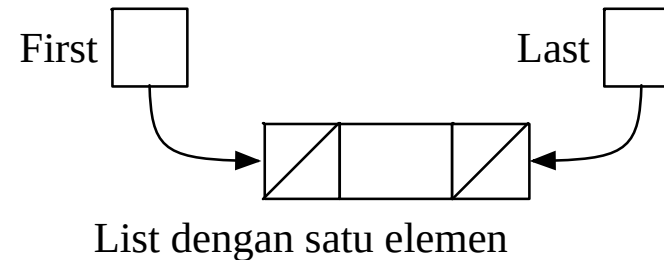
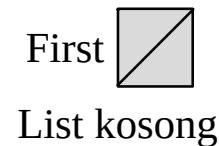
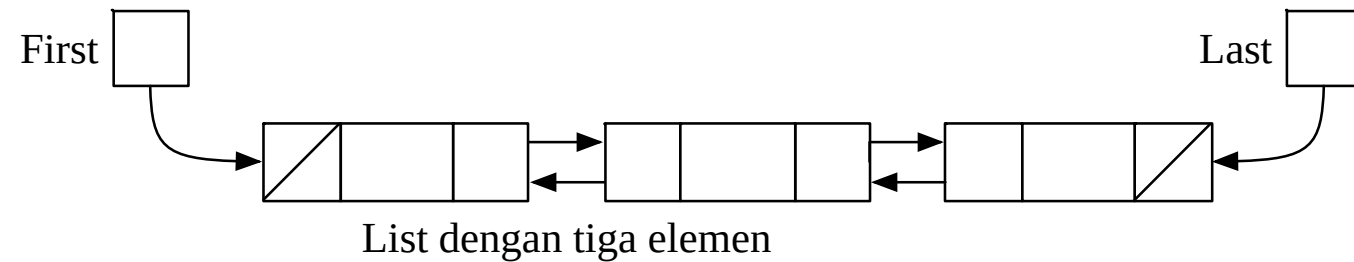
List dengan satu elemen

List dengan Pointer Ganda + Pencatatan Last

Elemen pertama: First(L)

Elemen terakhir: Last(L); Next(Last(L)) = Nil

List kosong: First(L) = Last (L) = Nil



Beberapa catatan

Dibutuhkan jika harus dilakukan banyak operasi terhadap elemen suksesor dan juga predesesor.

Dengan tersedianya alamat predesesor pada setiap elemen list, maka memorisasi Prec pada beberapa algoritma yang pernah ditulis dapat dihindari

Operasi dasar menjadi sangat “banyak”

Memori yang dibutuhkan membesar

Jika list logik semacam ini direpresentasi secara kontigu dengan tabel, maka sangat menguntungkan karena memorisasi Prev dan Next dilakukan dengan kalkulasi

Bahasan

Buatlah ADT list dengan elemen berpointer ganda dilengkapi driver:

- Representasi fisik: berkait dengan pointer
- Penunjuk First dan Last

Rep. Fisik dengan Pointer

```
#define NIL NULL
typedef int ElType;
typedef struct node *Address;
typedef struct node {
    ElType info;
    Address prev;
    Address next;
} Node;
/* Definisi list: */
/* List kosong: First = Nil and Last = Nil */
/* Setiap elemen dengan address P dapat diacu Info(P), Prev(P), Next(P) */
typedef struct {
    Address first; Address last;
} List;
/* Selektor */
#define INFO(p) (p)->info
#define PREV(p) (p)->prev
#define NEXT(p) (p)->next
#define FIRST(l) ((l).first)
#define LAST(l) ((l).last)
```

Beberapa primitif

Buatlah sebagai latihan:

```
void insertFirst(List *l, ElType x);  
/* I.S. List l terdefinisi */  
/* F.S. Menambahkan elemen baru x sebagai elemen pertama */  
  
void insertLast(List *l, ElType x);  
/* I.S. List l terdefinisi */  
/* F.S. x ditambahkan sebagai elemen terakhir l yang baru */
```

Beberapa primitif

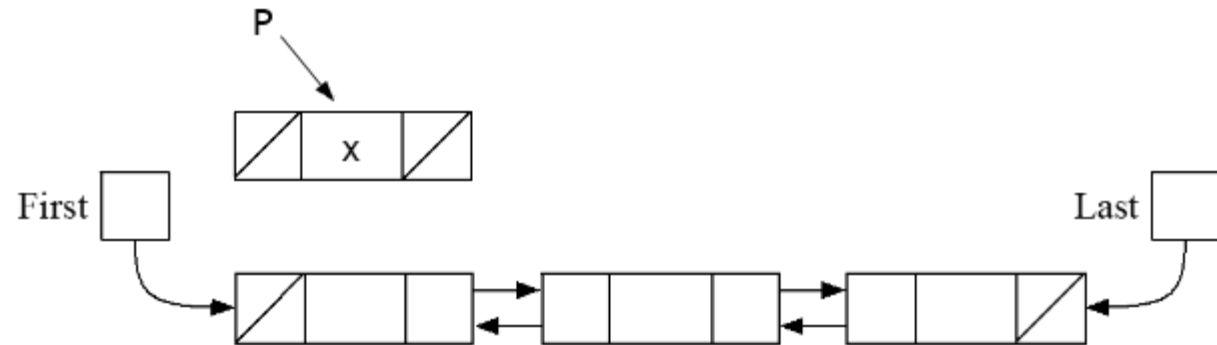
Buatlah sebagai latihan:

```
void deleteFirst(List *l, ElType *x);
/* I.S. List l tidak kosong */
/* F.S. x adalah elemen pertama list l sebelum penghapusan */
/*      Elemen list berkurang satu (mungkin menjadi kosong) */
/*      First element yg baru adalah suksesor elemen pertama yang lama */

void deleteLast(List *l, ElType *x) {
/* I.S. List l tidak kosong */
/* F.S. x adalah elemen terakhir list l sebelum penghapusan */
/*      Elemen list berkurang satu (mungkin menjadi kosong) */
/*      Last element baru adalah predesesor elemen pertama yg lama, jika ada */
```

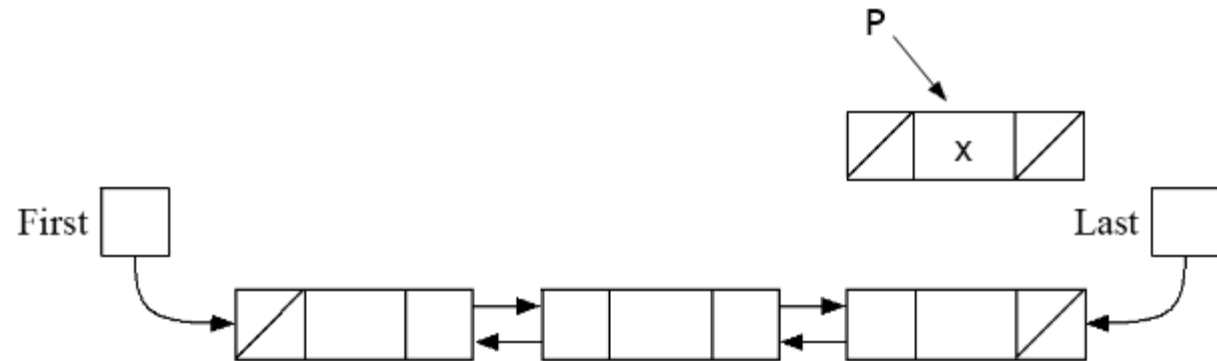
Beberapa primitif

```
void insertFirst(List *l, ElType x);  
/* I.S. List l terdefinisi */  
/* F.S. Menambahkan elemen baru x sebagai elemen pertama */  
/* Kamus Lokal */  
address p;  
  
/* Algoritma */  
p = newNode(x);  
if (p != NIL) {  
    NEXT(p) = FIRST(*l);  
    if (!isEmpty(*l)) {  
        PREV(FIRST(*l)) = p;  
    } else /* l kosong */ {  
        LAST(*l) = p;  
    }  
    FIRST(*l) = p;  
}  
}
```



Beberapa primitif

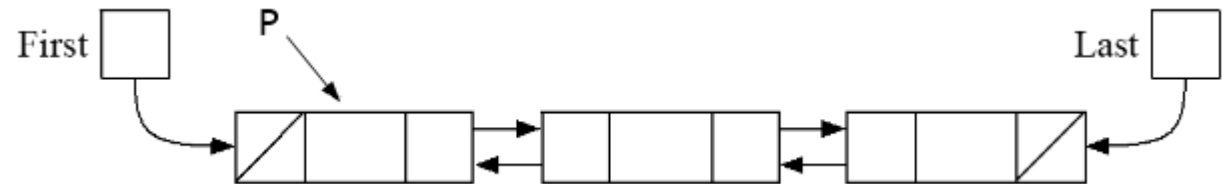
```
void insertLast(List *l, ElType x);  
/* I.S. List l terdefinisi */  
/* F.S. x ditambahkan sebagai elemen terakhir l yang baru */  
/* Kamus Lokal */  
address p;  
  
/* Algoritma */  
p = newNode(x);  
if (p != NIL) {  
    PREV(p) = LAST(*l);  
    if (!isEmpty(*l)) {  
        NEXT(LAST(*l)) = p;  
    } else /* l kosong */ {  
        FIRST(*l) = p;  
    }  
    LAST(*l) = p;  
}  
}
```



Beberapa primitif

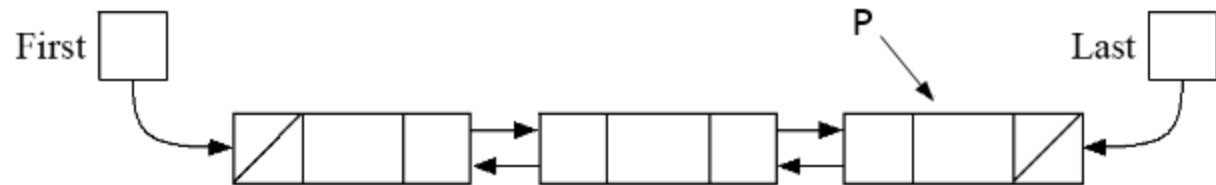
```
void deleteFirst(List *l, ElType *x);
/* I.S. List l tidak kosong */
/* F.S. x adalah elemen pertama list l sebelum penghapusan */
/*      Elemen list berkurang satu (mungkin menjadi kosong) */
/*      First element yg baru adalah suksesor elemen pertama yang lama */
/* Kamus Lokal */
address p;

/* Algoritma */
p = FIRST(*l);
*x = INFO(p);
if (FIRST(*l) == LAST(*l)) { /* l hanya 1 elemen */
    LAST(*l) = NIL;
} else { /* l > 1 elemen */
    PREV(NEXT(FIRST(*l))) = NIL;
}
FIRST(*l) = NEXT(FIRST(*l));
free(p);
}
```



Beberapa primitif

```
void deleteLast(List *l, ElType *x) {  
    /* I.S. List l tidak kosong */  
    /* F.S. x adalah elemen terakhir list l sebelum penghapusan */  
    /*      Elemen list berkurang satu (mungkin menjadi kosong) */  
    /*      Last element baru adalah predesesor elemen pertama yg lama, jika ada */  
    /* Kamus Lokal */  
    address p;  
  
    /* Algoritma */  
    p = LAST(*l);  
    *x = INFO(p);  
    if (FIRST(*l) == LAST(*l)) { /* l hanya 1 elemen */  
        FIRST(*l) = NIL;  
    } else { /* L > 1 elemen */  
        NEXT(PREV(LAST(*l))) = NIL;  
    }  
    LAST(*l) = PREV(LAST(*l));  
    free(p);  
}
```



Variasi List Linier

List Sirkuler

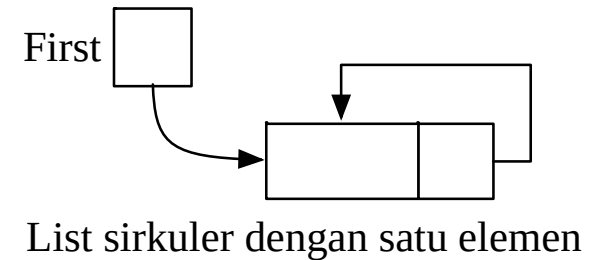
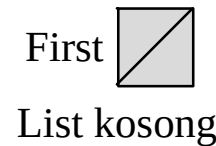
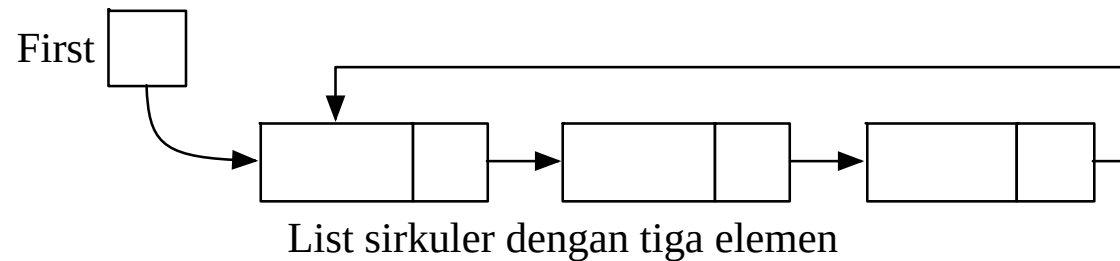
IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

List Sirkuler

Elemen pertama: $\text{First}(L)$

Elemen terakhir: $\text{Last}(L) = P$; $\text{Next}(P) = \text{First}(L)$

List kosong: $\text{First}(L) = \text{Nil}$



Beberapa catatan

List dengan representasi ini sebenarnya tidak mempunyai “First”

First adalah “Current Pointer”

Representasi ini dipakai jika dilakukan proses terus menerus terhadap anggota list (misalnya dalam round robin services pada sistem operasi)

Penambahan dan penghapusan pada elemen pertama akan berakibat harus melakukan traversal untuk mengubah Next dari elemen Last.

Bahasan - 3

Buatlah ADT list sirkuler + driver:

- Penunjuk First
- Representasi fisik: berkait dengan pointer

Rep. Fisik dengan Pointer

```
#define NIL NULL

typedef int ElType;
typedef struct node* Address;
typedef struct node { ElType info; address next; } Node;

/* Definisi list: */
/* List kosong: FIRST(l) = NIL */
/* Setiap elemen dengan address p dapat diacu INFO(p), NEXT(p) */
/* Elemen terakhir list: jika addressnya last,
   maka NEXT(last) = FIRST(l) */
typedef struct {
    address first;
} List;

/* Selektor */
#define INFO(p) (p)->info
#define NEXT(p) (p)->next
#define FIRST(l) ((l).first)
```

Beberapa primitif

Buatlah sebagai latihan:

```
boolean addrSearch(List l, Address p);  
/* Mencari apakah ada elemen list l yang beralamat p */  
/* Mengirimkan true jika ada, false jika tidak ada */  
void insertFirst(List *l, ElType x);  
/* I.S. List l terdefinisi */  
/* F.S. Menambahkan elemen bernilai x sebagai elemen pertama */  
void insertLast(List *l, ElType x);  
/* I.S. List l terdefinisi */  
/* F.S. x ditambahkan sebagai elemen terakhir l yang baru */
```

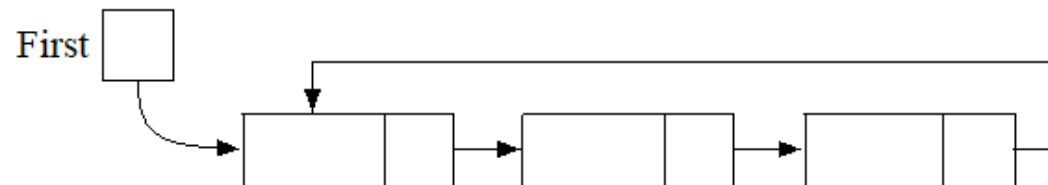
Beberapa primitif

Buatlah sebagai latihan:

```
void deleteFirst(List *l, ElType *x);
/* I.S. List l tidak kosong */
/* F.S. x adalah elemen pertama list l sebelum penghapusan */
/*      Elemen list berkurang satu (mungkin menjadi kosong) */
/*      First element yg baru adalah suksesor elemen pertama yang lama */
void deleteLast(List *l, ElType *x);
/* I.S. List l tidak kosong */
/* F.S. x adalah elemen terakhir list sebelum penghapusan */
/*      Elemen list berkurang satu (mungkin menjadi kosong) */
/*      Last element baru adalah predesesor elemen pertama yg lama, jika ada */
void displayList(List l);
/* I.S. List l mungkin kosong */
/* F.S. Jika list tidak kosong, semua nilai (info) yg disimpan pada elemen list diprint */
/*      Jika list kosong, hanya menuliskan "list kosong" */
```

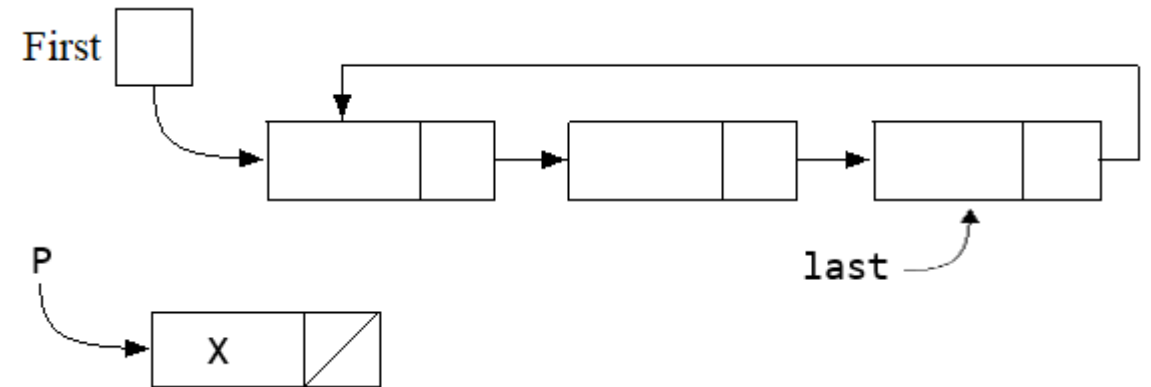
Beberapa primitif

```
boolean addrSearch(List l, Address p) {  
  /* Mencari apakah ada elemen list l yang beralamat p */  
  /* Mengirimkan true jika ada, false jika tidak ada */  
  /* Kamus Lokal */  
  Address pt;  
  /* Algoritma */  
  if (isEmpty(l)) {  
    return false;  
  } else {  
    pt = FIRST(l);  
    while ((NEXT(pt) != FIRST(l)) && (pt != p)) {  
      pt = NEXT(pt);  
    }  
    /* NEXT(pt) = FIRST(l) or pt = p */  
    return pt == p;  
  }  
}
```



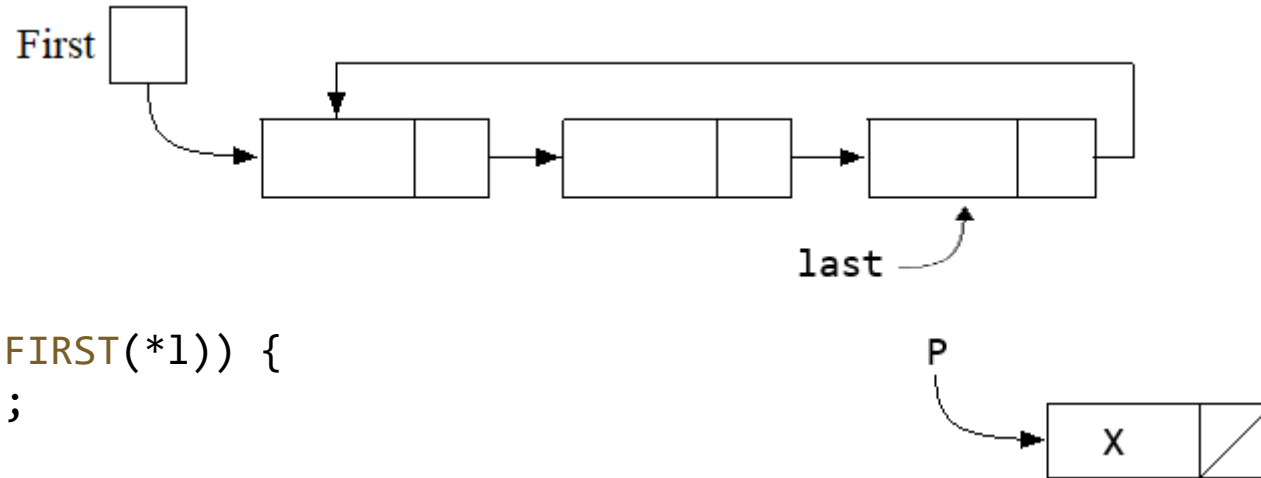
Beberapa primitif

```
void insertFirst(List *l, ElType x) {  
    /* I.S. List l terdefinisi */  
    /* F.S. Menambahkan elemen bernilai x sebagai elemen pertama l */  
    /* Kamus Lokal */  
    address p, last;  
    /* Algoritma */  
    p = newNode(x);  
    if (p != NIL) {  
        if (isEmpty(*l)) {  
            NEXT(p) = p;  
        } else /* *l tidak kosong */ {  
            last = FIRST(*l);  
            while (NEXT(last) != FIRST(*l)) {  
                last = NEXT(last);  
            } /* NEXT(last) = FIRST(*l) ==> elemen terakhir */  
            NEXT(p) = FIRST(*l);  
            NEXT(last) = p;  
        }  
        FIRST(*l) = p;  
    }  
}
```



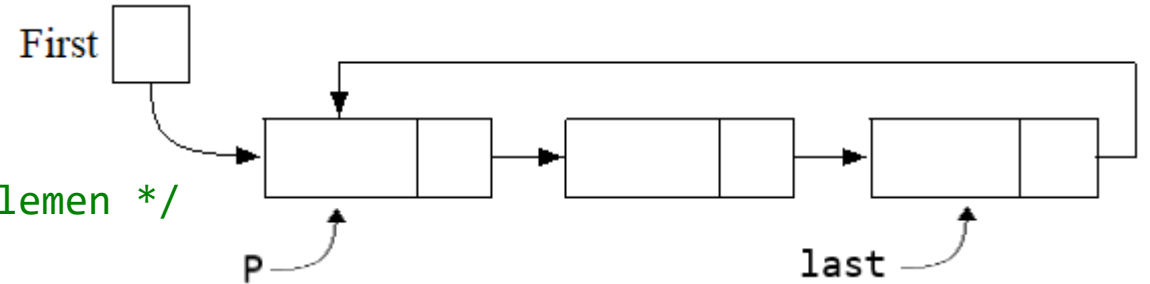
Beberapa primitif

```
void insertLast(List *l, ElType x) {  
    /* I.S. List l terdefinisi */ /* F.S. x ditambahkan sebagai elemen terakhir l yang baru */  
    /* Kamus Lokal */  
    address p, last;  
    /* Algoritma */  
    if (isEmpty(*l)) {  
        insertFirst(l,x);  
    } else {  
        p = newNode(x);  
        if (p != NIL) {  
            last = FIRST(*l);  
            while (NEXT(last) != FIRST(*l)) {  
                last = NEXT(last);  
            }  
            /* NEXT(last) = FIRST(*l) */  
            NEXT(last) = p;  
            NEXT(p) = FIRST(*l);  
        }  
    }  
}
```



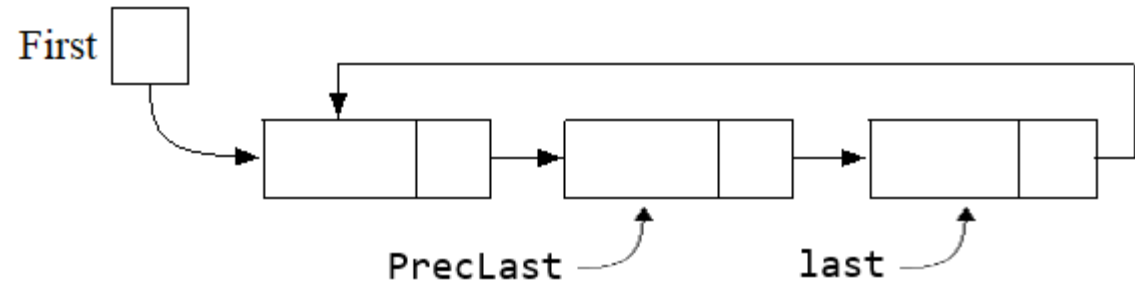
Beberapa primitif

```
void deleteFirst(List *l, ElType *x) {  
    /* I.S. List tidak kosong */  
    /* F.S. x adalah elemen pertama list l sebelum penghapusan */  
    /* Elemen list berkurang satu (mungkin menjadi kosong) */  
    /* First element yg baru adalah suksesor elemen pertama yang lama */  
    /* Kamus Lokal */  
    address p, last;  
    /* Algoritma */  
    p = FIRST(*l); *x = INFO(p);  
    if (NEXT(FIRST(*l)) == FIRST(*l)) { /* satu elemen */  
        FIRST(*l) = NIL;  
    } else {  
        last = FIRST(*l);  
        while (NEXT(last) != FIRST(*l)) {  
            last = NEXT(last);  
        }  
        /* NEXT(last) = FIRST(*l) */  
        FIRST(*l) = NEXT(FIRST(*l)); NEXT(last) = FIRST(*l);  
    }  
    free(p);  
}
```



Beberapa primitif

```
void deleteLast(List *l, ElType *x) {  
    /* I.S. List l tidak kosong */  
    /* F.S. x adalah elemen terakhir list l sebelum penghapusan */  
    /* Elemen list berkurang satu (mungkin menjadi kosong) */  
    /* Last element baru adalah predesesor elemen pertama yg lama, jika ada */  
    /* Kamus Lokal */  
    address last, preLast;  
    /* Algoritma */  
    last = FIRST(*l); preLast = NIL;  
    while (NEXT(last) != FIRST(*l)) {  
        preLast = last; last = NEXT(last);  
    } /* NEXT(last) = FIRST(*l) */  
    if (preLast == NIL) { /* kasus satu elemen */  
        FIRST(*l) = NIL;  
    } else {  
        NEXT(preLast) = FIRST(*l);  
    }  
    *x = INFO(last);  
    free(last);  
}
```



Beberapa primitif

```
void displayList(List l) {  
    /* I.S. List l mungkin kosong */  
    /* F.S. Jika list tidak kosong, semua nilai (info) yg disimpan pada elemen list diprint */  
    /*      Jika list kosong, hanya menuliskan "list kosong" */  
    /* Kamus Lokal */  
    address p;  
  
    /* Algoritma */  
    if (isEmpty(l)) {  
        printf("List Kosong \n");  
    } else {  
        p = FIRST(l);  
        printf("List: \n");  
        do {  
            printf("%d \n", INFO(p));  
            p = NEXT(p);  
        } while (p != FIRST(l));  
    }  
}
```

Latihan Soal Variasi List Linier

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Soal No. 1

Buatlah prosedur **insertFirst** untuk sebuah list *l* dengan *l* adalah List yang mencatat elemen pertama (First) dan elemen terakhir (Last)

procedure insertFirst(input/output *l*:List, input *x*:ElType)

List *l* mungkin kosong.

Soal No. 2

Buatlah prosedur **insertLast** dari sebuah list l dengan l adalah List yang mencatat elemen pertama (First) dan elemen terakhir (Last)

procedure insertLast(input/output l:List, input x:ElType)

List l mungkin kosong.

Soal No. 3

Buatlah fungsi **search** untuk mengetahui apakah sebuah nilai x terdapat dalam sebuah list l . l adalah List yang mencatat elemen pertama (First) dan elemen terakhir (Last) dan elemen terakhir adalah dummy.

function search(l :List, x :ElType) → boolean

List l mungkin kosong

Stack dengan Struktur Berkait

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Stack

Stack, sederetan elemen yang:

dikenali elemen puncaknya (Top)

aturan penambahan dan penghapusan elemennya tertentu:

- Penambahan selalu dilakukan "di atas" Top
- Penghapusan selalu dilakukan pada Top

Top adalah satu-satunya lokasi terjadinya operasi

Elemen Stack tersusun secara LIFO (Last In First Out)

Definisi operasi

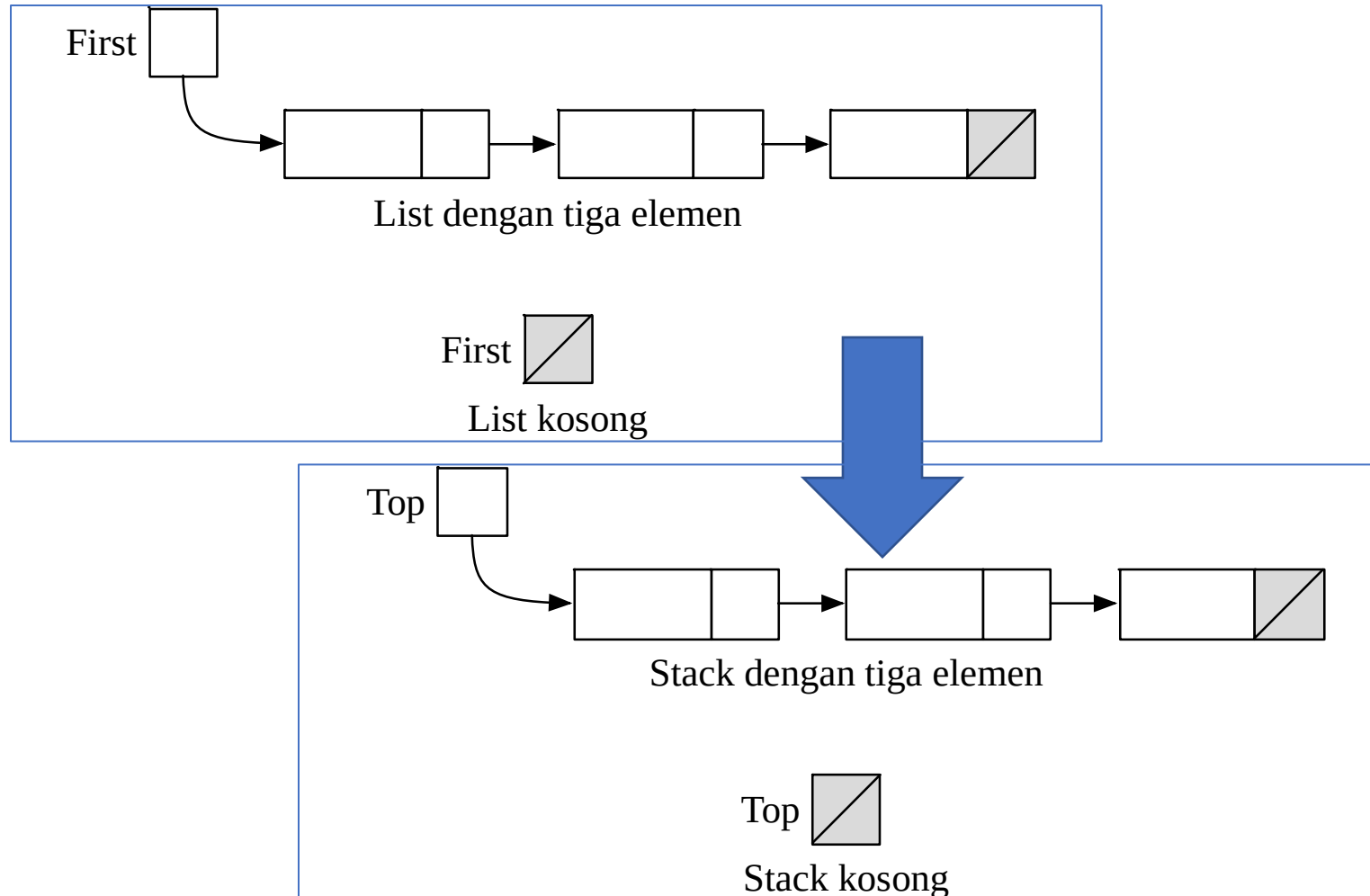
Jika diberikan S adalah Stack dengan elemen $ElmtS$

$CreateStack: \rightarrow S$	{ Membuat sebuah tumpukan kosong }
$top: S \rightarrow ElmtS$	{ Mengirimkan elemen teratas S saat ini }
$length: S \rightarrow \underline{integer}$	{ Mengirimkan banyaknya elemen S saat ini }
$push: ElmtS \times S \rightarrow S$	{ Menambahkan sebuah elemen $ElmtS$ sebagai TOP, TOP berubah nilainya }
$pop: S \rightarrow S \times ElmtS$	{ Mengambil nilai elemen TOP, sehingga TOP yang baru adalah elemen yang datang sebelum elemen TOP, mungkin S menjadi kosong }
$isEmpty: S \rightarrow \underline{boolean}$	{ Test stack kosong, true jika S kosong, false jika S tidak kosong }

Representasi berkait seperti apa yang paling cocok untuk stack?

Stack dengan Struktur Berkait

List linier “biasa” $\approx$ stack



Operasi-Operasi Dasar pada Stack

CreateStack $\approx$ CreateList

push $\approx$ insertFirst

pop $\approx$ deleteFirst

length $\approx$ length

isEmpty $\approx$ isEmpty

ADT Stack Representasi List

```
/* File: stack_linked.h */
#ifndef STACK_LINKED_H
#define STACK_LINKED_H
#include "boolean.h"
#include <stdlib.h>

#define NIL NULL
/* Deklarasi infotype */
typedef int ElType;
/* Stack dengan representasi berkait dengan pointer */
typedef struct node* Address;
typedef struct node {
    ElType info;
    address next;
} Node;

/* Type stack dengan ciri Top: */
typedef struct {
    address addrTop; /* alamat Top: elemen puncak */
} Stack;
```

ADT Stack Representasi List

```
/* Selektor */
#define      NEXT(p) (p)->next
#define      INFO(p) (p)->info
#define ADDR_TOP(s) (s).addrTop
#define      TOP(s) (s).addrTop->Info

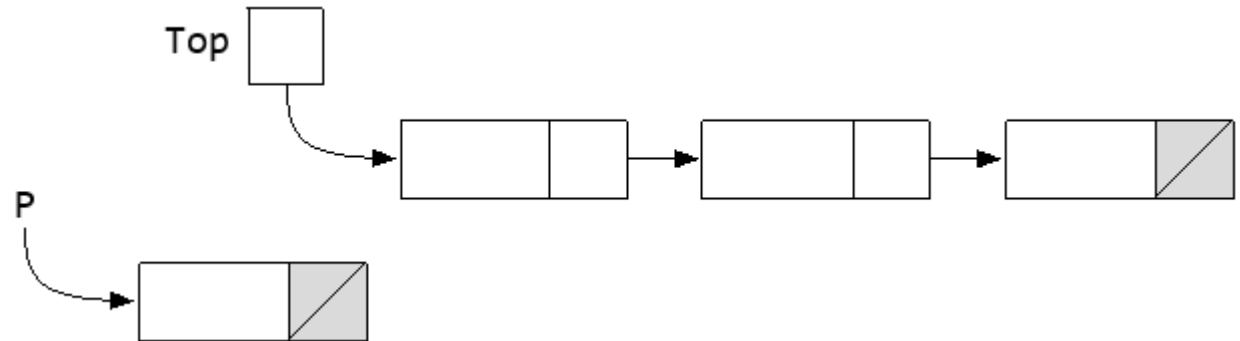
/* Prototype manajemen memori */
Address newNode(ElType x);
/* Mengembalikan alamat sebuah Node hasil alokasi dengan info = x,
   atau NIL jika alokasi gagal */
```

ADT Stack Representasi List

```
/* ***** PROTOTYPE REPRESENTASI LOJIK STACK ***** */
boolean isEmpty(Stack s);
/* Mengirim true jika Stack kosong: TOP(s) = NIL */
void CreateStack(Stack *s);
/* I.S. sembarang */
/* F.S. Membuat sebuah stack s yang kosong */
void push(Stack *s, ElType x);
/* Menambahkan x sebagai elemen Stack s */
/* I.S. s mungkin kosong, x terdefinisi */
/* F.S. x menjadi Top yang baru jika alokasi x berhasil, */
/*      jika tidak, s tetap */
/* Pada dasarnya adalah operasi insertFirst pada list linier */
void pop(Stack *s, ElType *x);
/* Menghapus Top dari Stack s */
/* I.S. s tidak kosong */
/* F.S. x adalah nilai elemen Top yang lama, */
/*      elemen Top yang lama didealokasi */
/* Pada dasarnya adalah operasi deleteFirst pada list linier */
#endif
```

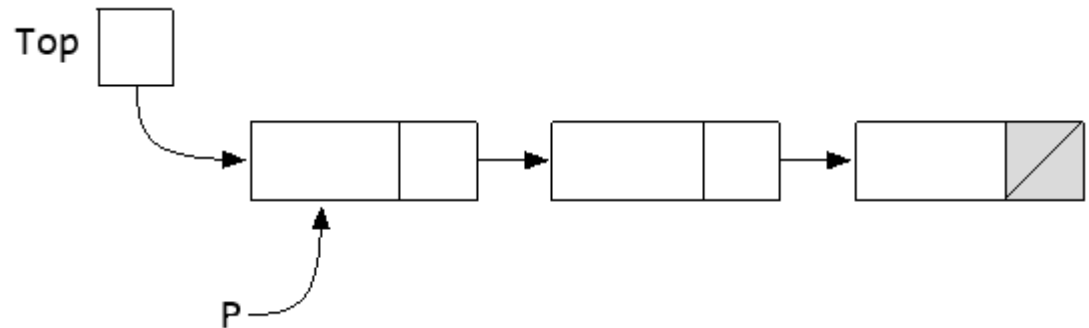
ADT Stack Representasi List

```
void push(Stack *s, ElType x);  
/* Menambahkan x sebagai elemen Stack s */  
/* I.S. s mungkin kosong, x terdefinisi */  
/* F.S. x menjadi Top yang baru jika alokasi x berhasil, */  
/*      jika tidak, s tetap */  
/* Pada dasarnya adalah operasi insertFirst pada list linier */  
  
/* Kamus Lokal */  
address p;  
  
/* Algoritma */  
p = newNode(x);  
if (p != NIL) {  
    NEXT(p) = ADDR_TOP(*s);  
    ADDR_TOP(*s) = p;  
} /* else: alokasi gagal, s tetap */  
}
```



ADT Stack Representasi List

```
void pop(Stack *s, ElType *x);  
/* Menghapus Top dari Stack s */  
/* I.S. s tidak mungkin kosong */  
/* F.S. x adalah nilai elemen Top yang lama, */  
/*      elemen Top yang lama didealokasi */  
/* Pada dasarnya adalah operasi deleteFirst pada list linier */  
  
/* Kamus Lokal */  
address p;  
  
/* Algoritma */  
*x = TOP(*s);  
p = ADDR_TOP(*s);  
ADDR_TOP(*s) = NEXT(ADDR_TOP(*s));  
NEXT(p) = NIL;  
free(p);  
}
```



Queue dengan Struktur Berkait

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Queue



Queue adalah sederetan elemen yang:

- dikenali elemen pertama (HEAD) dan elemen terakhirnya (TAIL).
- aturan penambahan dan penghapusan elemennya didefinisikan sebagai berikut:
Penambahan selalu dilakukan setelah **elemen terakhir**,
Penghapusan selalu dilakukan pada **elemen pertama**.

Definisi operasi

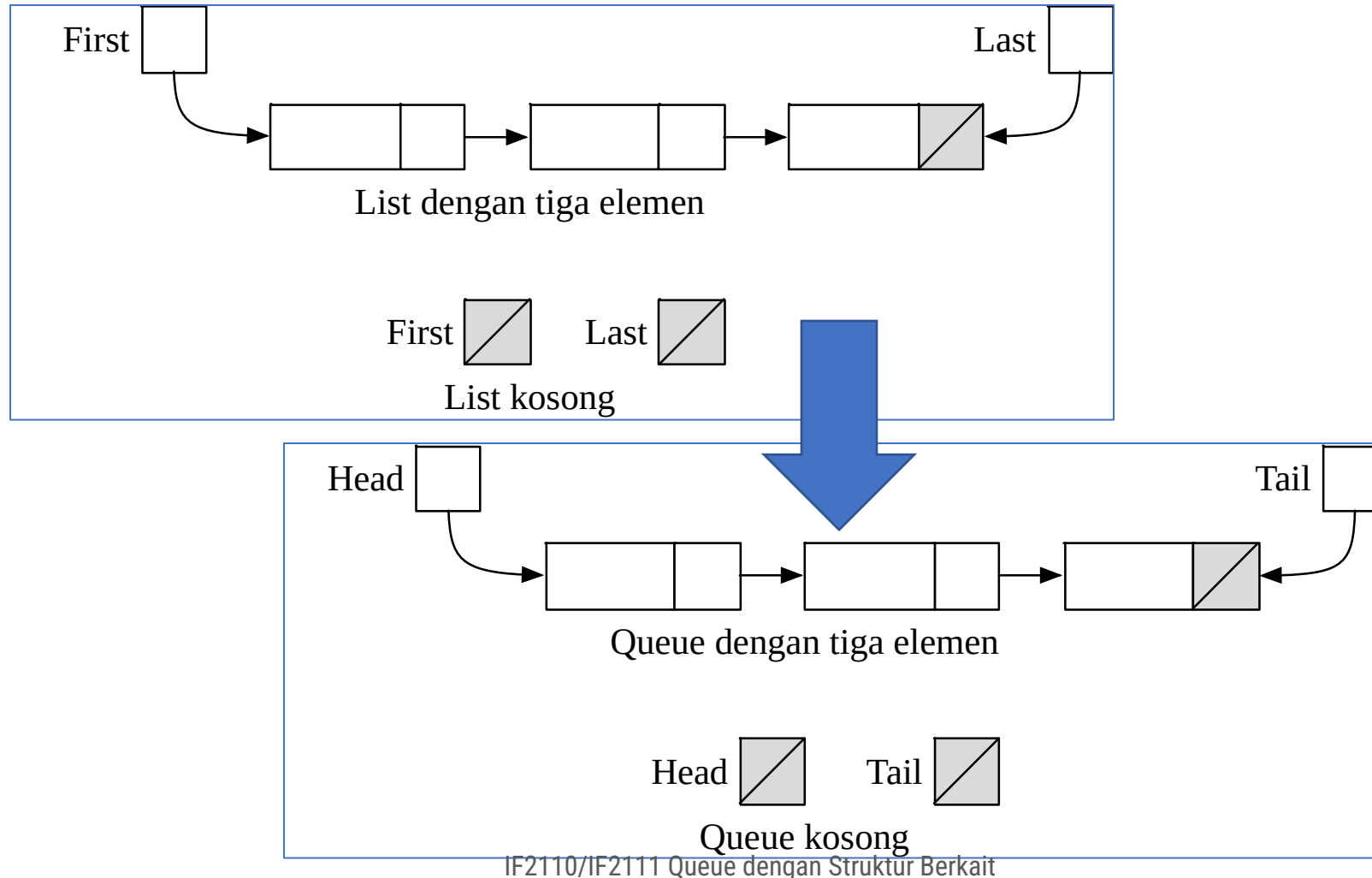
Jika diberikan Q adalah Queue dengan elemen $ElmtQ$

$CreateQueue: \rightarrow Q$	{ Membuat sebuah antrian kosong }
$head: Q \rightarrow ElmtQ$	{ Mengirimkan elemen terdepan Q saat ini }
$length: Q \rightarrow \underline{integer}$	{ Mengirimkan banyaknya elemen Q saat ini }
$enqueue: ElmtQ \times Q \rightarrow Q$	{ Menambahkankan sebuah elemen setelah elemen paling belakang Queue }
$dequeue: Q \rightarrow Q \times ElmtQ$	{ Menghapus kepala Queue, mungkin Q menjadi kosong }
$isEmpty: Q \rightarrow \underline{boolean}$	{ Tes terhadap Q : true jika Q kosong, false jika Q tidak kosong }

Representasi berkait seperti apa yang paling cocok untuk queue?

Queue dengan Struktur berkait

List Linier yang dicatat first dan last $\approx$ queue



Operasi-operasi dasar pada Queue

CreateQueue $\asymp$ CreateList

enqueue $\asymp$ insertLast

dequeue $\asymp$ deleteFirst

length $\asymp$ length

isEmpty $\asymp$ isEmpty

ADT Queue dengan Rep. List

```
/* File: queue_linked.h */
#ifndef QUEUE_LINKED_H
#define QUEUE_LINKED_H
#include "boolean.h"
#include <stdlib.h>

#define NIL NULL
/* Deklarasi infotype */
typedef int ElType;
/* Queue dengan representasi berkait dengan pointer */
typedef struct node* Address;
typedef struct node {
    ElType info;
    address next;
} Node;

/* Type queue dengan ciri HEAD dan TAIL: */
typedef struct {
    address addrHead; /* alamat penghapusan */
    address addrTail; /* alamat penambahan */
} Queue;
```

ADT Queue dengan Rep. List

```
/* Selektor */
#define NEXT(p) (p)->next
#define INFO(p) (p)->info

#define ADDR_HEAD(q) (q).addrHead
#define ADDR_TAIL(q) (q).addrTail
#define HEAD(q) (q).addrHead->info

/* Prototype manajemen memori */
Address newNode(ElType x);
/* Mengembalikan alamat sebuah Node hasil alokasi dengan info = x,
   atau NIL jika alokasi gagal */
```

ADT Queue dengan Rep. List

```
boolean isEmpty(Queue q);
/* Mengirim true jika q kosong: ADDR_HEAD(q)=NIL and ADDR_TAIL(q)=NIL */
int length(Queue q);
/* Mengirimkan banyaknya elemen queue. Mengirimkan 0 jika q kosong */

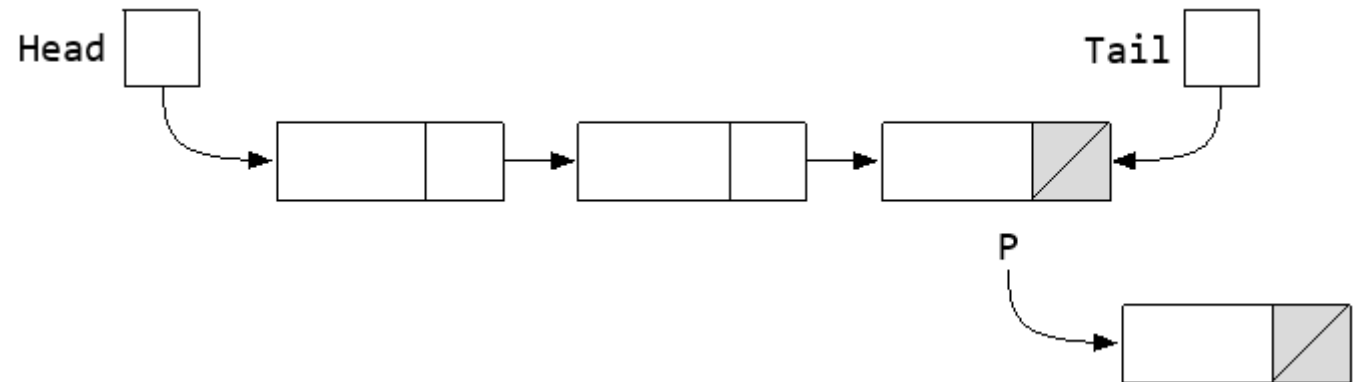
/** Kreator */
void CreateQueue(Queue *q);
/* I.S. sembarang */
/* F.S. Sebuah q kosong terbentuk */

/** Primitif Enqueue/Dequeue */
void enqueue(Queue *q, ElType x);
/* Proses: Mengalokasi x dan menambahkan x pada bagian Tail dari q
   jika alokasi berhasil; jika alokasi gagal q tetap */
/* Pada dasarnya adalah proses insertLast */
/* I.S. q mungkin kosong */
/* F.S. x menjadi Tail, Tail "maju" */
void dequeue(Queue *q, ElType *x);
/* Proses: Menghapus x pada bagian HEAD dari q dan mendealokasi elemen HEAD */
/* Pada dasarnya operasi deleteFirst */
/* I.S. q tidak mungkin kosong */
/* F.S. x = nilai elemen HEAD pd I.S., HEAD "mundur" */
#endif
```

ADT Queue dengan Rep. List

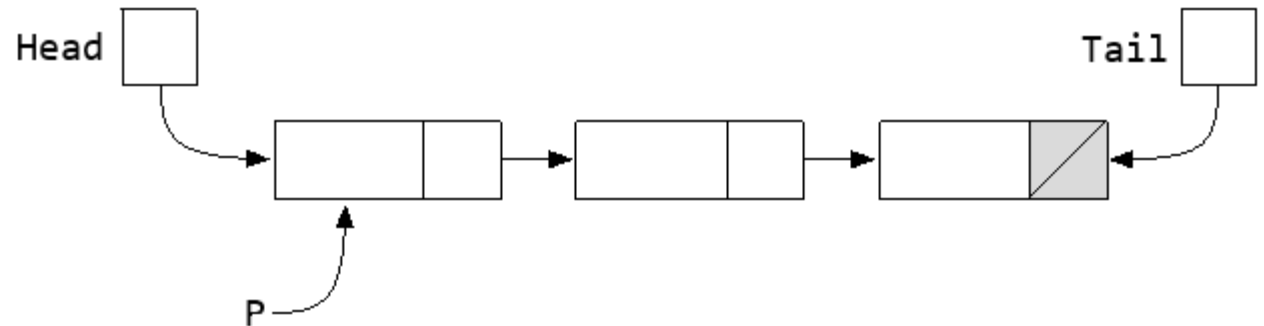
```
void enqueue(Queue *q, ElType x) {  
    /* Proses: Mengalokasi x dan menambahkan x pada bagian Tail dari q  
       jika alokasi berhasil; jika alokasi gagal q tetap */  
    /* Pada dasarnya adalah proses insertLast */  
    /* I.S. q mungkin kosong */  
    /* F.S. x menjadi Tail, Tail "maju" */
```

```
    /* Kamus Lokal */  
    address p;  
    /* Algoritma */  
    p = newNode(x);  
    if (p != NIL) {  
        if (isEmpty(*q)) {  
            ADDR_HEAD(*q) = p;  
        } else {  
            NEXT(ADDR_TAIL(*q)) = p;  
        }  
        ADDR_TAIL(*q) = p;  
    } /* else: alokasi gagal, q tetap */  
}
```



ADT Queue dengan Rep. List

```
void dequeue(Queue *q, ElType *x) {  
    /* Proses: Menghapus x pada bagian HEAD dari q dan mendealokasi  
       elemen HEAD */  
    /* Pada dasarnya operasi delete first */  
    /* I.S. q tidak mungkin kosong */  
    /* F.S. x = nilai elemen HEAD pd I.S., HEAD "mundur" */  
  
    /* Kamus Lokal */  
    address p;  
    /* Algoritma */  
    *x = HEAD(*q);  
    p = ADDR_HEAD(*q);  
    ADDR_HEAD(*q) = NEXT(ADDR_HEAD(*q));  
    if (ADDR_HEAD(*q) == NIL) {  
        ADDR_HEAD(*q) = NIL;  
    }  
    NEXT(p) = NIL;  
    free(p);  
}
```



Priority Queue

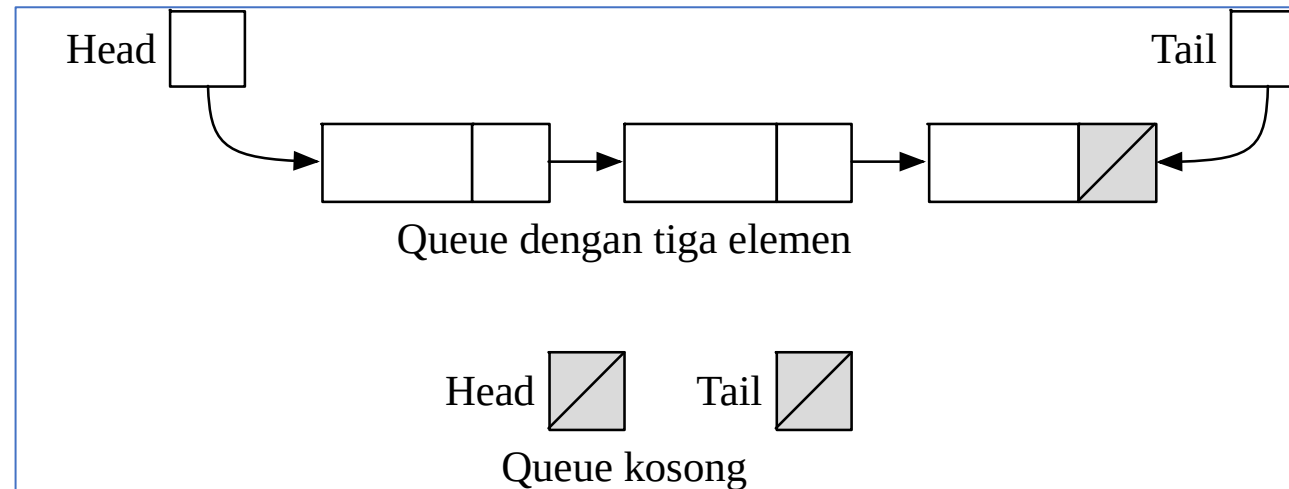
Priority queue:

- Elemen queue terurut menurut suatu prioritas tertentu
- Sering dianggap sebagai *modified queue*
- Menambahkan elemen berarti menambahkan elemen sesuai urutan prioritas
- Menghapus elemen adalah menghapus elemen dengan prioritas tertinggi/terendah (pada bagian Head)

Representasi berkait seperti apa yang paling cocok untuk priority queue?

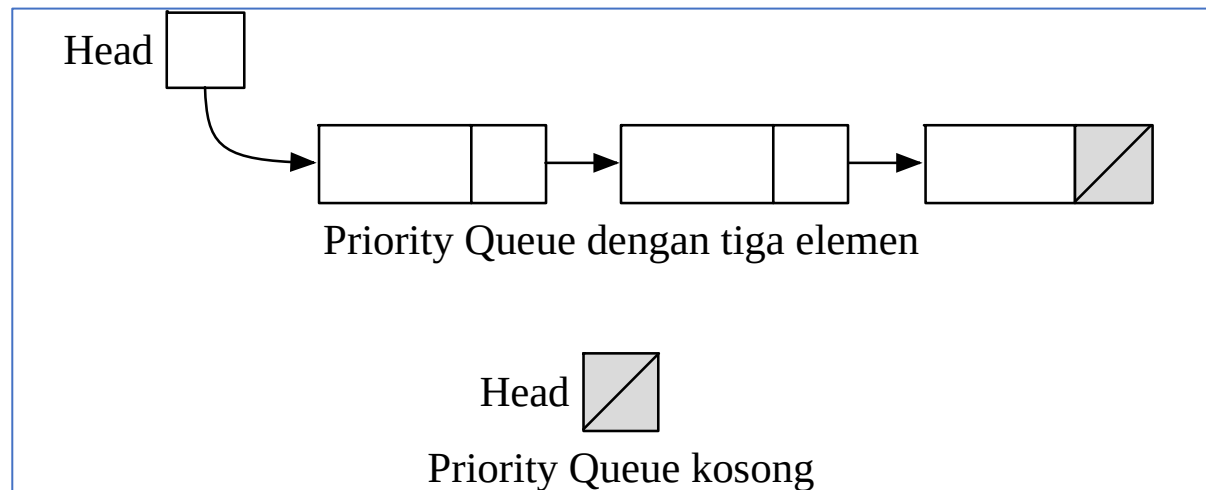
Priority Queue dengan Rep. List

List Linier yang dicatat first dan last $\approx$ queue “biasa”



Priority Queue dengan Rep. List

List linier “biasa” ~ priority queue



Operasi-operasi dasar Priority Queue

Node elemen queue:

```
type Node: < info: ElType,  
                prio: integer,  
                next: Address >
```

- Enqueue → menambahkan elemen secara terurut mengikuti prioritas tertentu
- Dequeue → menghapus elemen dengan prioritas tertinggi (yaitu di Head)
 - Pada dasarnya sama saja dengan Dequeue pada queue/list biasa

Perhatikan representasi logik yang digunakan

Latihan Soal

Queue dengan Struktur Berkait

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Latihan Soal Priority Queue

Implementasikan prosedur Enqueue dan Dequeue (dalam Bahasa C) pada priority queue sesuai dengan deskripsi berikut ini.

ADT Priority Queue dengan Rep. List

```
/* File: prioqueueelist.h */
#ifndef _PRIOQUEUEELIST_H
#define _PRIOQUEUEELIST_H
#include "boolean.h"
#include <stdlib.h>

#define Nil NULL
/* Deklarasi type elemen */
typedef int ElType;
/* Priority Queue dengan representasi berkait dengan pointer */
typedef struct tNode *address;
typedef struct tNode {
    ElType info;
    int prio;
    address next;
} Node;

/* Type PrioQueue dengan HEAD */
typedef struct {
    address addrHead; /* alamat elemen pertama */
} PrioQueue;
```

ADT Priority Queue dengan Rep. List

```
/* Selektor */
#define NEXT(p) (p)->next
#define INFO(p) (p)->info
#define PRIO(p) (p)->prio

#define ADDR_HEAD(q) (q).addrHead
#define HEAD(q) (q).addrHead->info

/* Prototype manajemen memori */
Address newNode(ElType x, int pr);
/* Mengembalikan alamat sebuah Node hasil alokasi dengan info = x dan prio = pr,
   atau NIL jika alokasi gagal */

void delNode(address P);
/* I.S. P adalah hasil alokasi, P != Nil */
/* F.S. Alamat P didealokasi, dikembalikan ke sistem */
```

ADT Priority Queue dengan Rep. List

```
boolean isEmpty(PrioQueue q);
/* Mengirim true jika q kosong: ADDR_HEAD(q)=NIL and ADDR_TAIL(q)=NIL */
int length(PrioQueue q);
/* Mengirimkan banyaknya elemen queue. Mengirimkan 0 jika q kosong */

/** Kreator */
void CreateQueue(PrioQueue *q);
/* I.S. sembarang */
/* F.S. Sebuah q kosong terbentuk */

/** Primitif Enqueue/Dequeue */
void enqueue(PrioQueue *q, ElType x, int pr);
/* Proses: Mengalokasi x dengan prio pr dan menambahkan x pada q
    jika alokasi berhasil; jika alokasi gagal q tetap */
/* I.S. q mungkin kosong */
/* F.S. x menjadi elemen q sesuai dengan urutan prioritas */

void dequeue(PrioQueue *q, ElType *x, int *pr);
/* Proses: Menghapus x pada bagian HEAD dari q dan mendealokasi elemen HEAD */
/* Pada dasarnya operasi deleteFirst */
/* I.S. q tidak mungkin kosong */
/* F.S. x = nilai elemen HEAD pd I.S., HEAD "mundur" */
#endif
```

Priority Queue dengan Rep. List

List linier “biasa” → priority queue

